الجمهورية العربية السورية

وزارة التعليم العالي

جامعة طرطوس

كلية هندسة تكنولوجيا المعلومات والاتصالات

قسم هندسة تكنولوجيا المعلومات

مشروع تخرج أُعِدَّ لِنيل درجة الإجازة في هندسة تكنولوجيا المعلومات والاتصالات

قسم هندسة تكنولوجيا المعلومات

بعنوان:

منصة تبادل مهارات لا مركزية بمصادقة بيومترية متكيفة وتوثيق عقود على البلوكشين

Decentralized Skill Exchange Platform: Adaptive Biometrics and Blockchain Contract Documentation

اعداد :

ماري نبيل إبراهيم منال ياسر عدره

رهف يونس عبد الحميد

بإشراف :

الدكتور المهندس رضوان دنده

بسم الله نبدأ وعليه نتوكل ما كان هذا العمل سينجز لولا توفيق الله ورعايته

بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ

{وَكَانَ فَضْلُ اللَّهِ عَلَيْكَ عَظِيمًا}

صَدَقَ اللَّهُ الْعَلِيُّ الْعَظِيمُ

الاهداء :

نهدي هذا العمل الى كل من ألهمنا ودعمنا في كل خطوة

الى كل من حول اللحظات الى محطات بارزة في المعرفة والتقدم

الى كل من غرس فينا الايمان وبإن الوقت ليس مجرد أيام تعد وتحصى بل أداة لبناء حياة أفضل

نهدي هذا البحث لعائلاتنا التي كانت وستبقى مصدر دعمنا الأول والدائم في كل مراحل حياتنا

نهديه لكل معلم وأستاذ ساهم في رسم مسارنا نحو المعرفة

الشكر والتقدير :

في البداية نحمد الله ونشكره على عظيم رزقه وتوفيقه وكرمه .

نود أن نعرب عن أمتناننا وتقديرنا العميق للدكتور المشرف

/ الدكتور رضوان دنده /

لتوجيهاته الأكاديمية ونصائحه القيمة والدعم الكامل المتواصل طوال فترة البحث وكتابة هذا المشروع

ونتقدم بجزيل الشكر للكادر الإداري والتعليمي في قسمنا منذ اليوم الأول

الغاية من المشروع

يهدف المشروع إلى :

- ❖ تمكين تبادل المهارات بشكل آمن: توفير منصة تفاعلية تسمح للمستخدمين (أصحاب المهارات والباحثين عنها) بالتواصل والتعاقد بشكل رقمي وآمن، مع ضمان هوية كل طرف من خلال تقنيات التحقق البيومتري المتقدمة.
- ❖ توثيق العقود باستخدام تقنية البلوكشين (Blockchain) : توثيق جميع العقود المبرمة بين الأطراف على شبكة البلوكشين (Sepolia Ethereu) مع تخزين البيانات عبر IPFS، مما يضمن عدم إمكانية التلاعب بالعقود أو إنكارها لاحقاً، مع إضافة طابع زمني موثوق (Trusted Timestamp)
- ❖ تطبيق نظام بصمة متعدد العوامل: دمج أربع وسائل تحقق بيومتري (بصمة الوجه، بصمة الصوت، بصمة الإصبع، والتوقيع الرقمي) مع نظام كشف حيوية (Liveness Detection) لمنع انتحال الهوية، ودعم تقنيات مثل Windows Hello و Suprema BioMini للمؤسسات.
- ❖ تسريع عملية إيجاد المهارات المناسبة باستخدام الذكاء الاصطناعي: تطبيق نظام توصيات ذكي يعتمد على خوارزمية FAISS (Facebook AI Similarity Search) لتحليل سلوك المستخدمين ومهاراتهم، واقتراح الأشخاص أو المشاريع المناسبة في وقت أقل من 0.5 ثانية، مع تقديم شرح مفهوم للتوصية (Explainable AI)
- ❖ توفير بيئة تواصل آمنة ومشفرة: إنشاء نظام محادثة فوري (Real-time Chat) مع تشفير كامل للرسائل (Encrypted Chat) باستخدام مفاتيح مشتقة لكل غرفة، ودعم رفع الملفات (صور، فيديو، مستندات) مع تخزينها على IPFS ونظام كشف الفيروسات.
- ❖ بناء نظام ثقة Trust Scoreذاتي التعلم: تطوير خوارزمية لحساب درجة ثقة كل مستخدم بناءً على إنجازاته (نسبة إتمام العقود، التقييمات، سرعة الاستجابة، عدد العقود المنفذة)، مما يخلق بيئة تفاعلية تحفز المستخدمين على الالتزام بجودة العمل.
- ❖ توفير لوحة تحكم متكاملة للمستخدمين: تصميم واجهة مستخدم تفاعلية تعرض إحصائيات المستخدم (عدد العقود النشطة، درجة الثقة، التقييمات)، مع إمكانية إدارة الملف الشخصي، رفع الصور، والتحقق من الهوية عبر الوثائق الرسمية.
- ❖ توسيع نطاق الوصول وضمان الأمان المؤسسي: دعم بنية Zero Trust Architecture مع نظام Circuit Breaker لحماية النظام من الهجمات، و Rate Limiting موزع باستخدام Redis لمنع إساءة الاستخدام، مع سجلات تدقيق كاملة (Audit Logsلجميع العمليات.

الملخص

يهدف هذا المشروع إلى بناء منصة إلكترونية ذكية لتبادل المهارات والعمل الحر (Freelancing)، تجمع بين الذكاء الاصطناعي، التحقق البيومتري، وتقنية البلوكشين لحل مشكلات الثقة وتوثيق الهوية الرقمية. تتيح المنصة للمستخدمين التواصل عبر نظام محادثة مشفر، وإنشاء عقود رقمية تُوقع بعد التحقق بيومترياً (الوجه، الصوت، بصمة الأصبع، والتوقيع الرقمي) مع ميزة كشف الحيوية لضمان الأمان ومنع التزوير. تم تطوير النظام بالاعتماد على Django REST Framework للـ Backend، و React.js للـ Frontend، بدعم من WebSockets للدردشة الفورية. كما دُمجت تقنيات متقدمة تشمل FAISS لنظام التوصيات، و DeepFace و SpeechBrain للمطابقة البيومترية، بالإضافة إلى مكتبة Web3.py لتوثيق العقود على شبكة Ethereum (Sepolia)، ونظام IPFS للتخزين اللامركزي. من الناحية المنهجية، تمت دراسة التطبيقات العالمية المشابهة مثل LinkedIn و Upwork و .Fiverrكما تم توثيق بنية النظام وتصميم واجهاته هندسياً باستخدام مخططات UML الشاملة (حالات الاستخدام، الفئات، التتابع الحركي Transitions، والتسلسل)، إلى جانب إجراء اختبارات أمان ناجحة ضد هجمات إعادة الهجوم (Replay Attack)والقوة الغاشمة (Brute Force) . يختتم المشروع بتقديم نظام برمجائي متكامل يعمل بكامل كفاءته، مع استعراض أبرز التحديات التي تم التغلب عليها كربط الـ WebSockets مع الـ JWT، وضبط عتبات التعرف البيومتري، وتحسين أداء .FAISSوأخيراً، طُرحت مقترحات للتطوير المستقبلي تتضمن إطلاق تطبيق جوال بتقنية Flutter، دعم Apple FaceID، وتوسيع المعمارية لاستيعاب أكثر من 10,000 مستخدم متزامن باستخدام Kubernetes و Microservices.

Abstract

This project aims to build an intelligent freelancing and skill-swapping platform. It integrates Artificial Intelligence, biometric authentication, and blockchain technology to address trust and digital identity verification issues. The platform features an encrypted chat system and digital contracts signed via multi-modal biometrics (face, voice, fingerprint, and digital signature) backed by liveness detection to prevent fraud.

The system architecture is developed using Django REST Framework for the backend, React.js for the frontend, and WebSockets for real-time messaging. Advanced integrations include FAISS for the recommendation system, DeepFace and SpeechBrain for biometric matching, Web3.py for documenting contracts on the Ethereum (Sepolia) network, and IPFS for decentralized storage.

Methodologically, global benchmarks such as LinkedIn, Upwork, and Fiverr were analyzed. The system architecture and interfaces were engineered and documented using comprehensive UML diagrams (Use Case, Class, Transition, and Sequence). Furthermore, successful security assessments were conducted against Replay and Brute Force attacks.

The project concludes with a fully functional software system. It highlights resolved technical challenges, including WebSockets-JWT integration, biometric threshold tuning, and FAISS optimization. Future enhancements propose launching a Flutter mobile application, supporting Apple FaceID, and scaling the architecture to handle over $10{,}000$ concurrent users using Kubernetes and Microservices

الفهرس

المحتويات

الغاية من المشروع 5

الملخص 6

الفصل الأول : 13

الإطار العام للمشروع 13

المقدمة : 14

بيان المشكلة : 14

أهداف المشروع : 14

نطاق المشروع : 14

تنظيم المشروع : 15

الفصل الثاني : 16

دراسة للتطبيقات المشابهة 16

المقدمة: 17

التطبيق الأول: 17

التطبيق الثاني : 19

التطبيق الثالث : 20

المقارنة : 22

الفصل الثالث : 24

الدراسة النظرية للمشروع 24

المقدمة : 25

- **المحور الأول: البيئة اللامركزية وبنية التشفير والحماية :** 25
- **المحور الثاني: المصادقة البيومترية العصبية ونظام التوصيات:** 28
- **المحور الثالث: بروتوكولات الاتصال وبيئة التطوير (الربط والبرمجة):** 31

الفصل الرابع: 34

دراسة تحليلية للمشروع 34

مقدمة : 35

مخطط حالة الاستخدام : 35

مخطط الصفوف : 39

مخطط التتابع : 45

قاعدة البيانات الخاصة بمشروعنا : 49

الفصل الخامس : 57

واجهات الموقع 57

مقدمة : 58

الواجهة الرئيسية : 58

واجهة التسجيل(Register) : 59

واجهة كشف إمكانيات الجهاز : 60

واجهة تسجيل الوجه (Face Recognition Enrollment) : 62

واجهة تسجيل الصوت : 63

واجهة التوقيع اليدوي : 64

واجهة تسجيل الدخول :..... 65
واجهة لوحة التحكم :..... 66
واجهة الملف الشخصي(Profile) :..... 69
واجهة تعديل الملف الشخصي edit :..... 72
واجهة المنشورات والمهارات :..... 73
واجهة غرفة الدردشة:..... 76
واجهة تفاصيل العقد :..... 78
واجهة إنشاء وتوقيع العقد :..... 79
الفصل السادس :..... 84
التحديات والاقتراحات المستقبلية..... 84
التحديات التي واجهتنا أثناء تنفيذ مشروعنا :..... 85
الاقتراحات المستقبلية لتطوير مشروعنا :..... 85
المراجع :..... 86

فهرس الاشكال

الشكل (1-2) أيقونة تطبيقLinkedIn 17

الشكل (2-2) أيقونة تطبيق Upwork..... 19

الشكل (2-3) أيقونة تطبيقFiverr 20

الشكل (4-4) مخطط حالة الاستخدام 36

الشكل (4-5) مخطط الصفوف..... 40

الشكل (4-6) مخطط التتابع 46

الشكل (4-7) مخطط التتابع الثاني والمخصص لعملية توثيق وتوقيع العقد الذكي..... 48

الشكل (4-8) قاعدة البيانات..... 50

الشكل(5-9) الواجهة الرئيسية..... 58

الشكل(5-11) واجهة كشف إمكانيات الجهاز..... 59

الشكل (5-12) واجهة إمكانيات الجهاز..... 60

الشكل (5-13) واجهة تسجيل الوجه 61

الشكل (5-14) واجهة تسجيل الوجه..... 62

الشكل (5-15) واجهة التوقيع اليدوي..... 64

الشكل (5-17) واجهة لوحة التحكم..... 65

الشكل (5-18) واجهة عقود المستخدم 66

الشكل (5-19) واجهة تفاصيل العقد 67

الشكل (5-20) واجهة تقييم العقد..... 68

الشكل (5-21) واجهة الملف الشخصي 68

الشكل (5-22) واجهة توضيحية لنجاح توثيق الحساب 69

الشكل (5-23) واجهة توثيق الهوية..... 70

الشكل (24-5) واجهة تعديل الملف الشخصي..... 71
الشكل (25-5) واجهة المهارات..... 72
الشكل (26-5) واجهة المنشورات..... 73
الشكل (27- 5) واجهة تفاصيل المنشور..... 74
الشكل (28-5) واجهة إنشاء منشور..... 74
الشكل (29-5) واجهة تعديل المنشور..... 75
الشكل (30-5) واجهة غرفة الدردشة..... 76
الشكل (31-5) واجهة تفاصيل العقد..... 76
الشكل (32-5) واجهة إنشاء وتوقيع العقد..... 78
الشكل (33-5) واجهة التحقق من الهوية بالبصمة البيومترية..... 79
الشكل (34-5) واجهة تحقق تكيفية..... 81
الشكل (35-5) واجهة اكتمال العقد..... 82
الشكل (36-5) واجهة التسجيل على البلوكشين..... 83

الفصل الأول :

الإطار العام للمشروع

Chapter One :

General Framework Of The Project

المقدمة :

في ظل التحول الرقمي المتسارع الذي يشهده العالم اليوم، أصبحت المنصات الإلكترونية الوسيلة الأساسية لربط أصحاب المهارات مع الباحثين عنها. تزايد الاعتماد على العمل الحر (Freelancing) بشكل كبير، حيث تشير الإحصاءات إلى أن سوق العمل الحر العالمي تجاوز حاجز الـ 15 مليار دولار سنوياً، مع توقعات بنمو مستمر.

بيان المشكلة :

على الرغم من انتشار منصات العمل الحر بشكل واسع، إلا أنها تعاني من مجموعة من المشاكل الجوهرية التي تحد من فعاليتها وتقلل من ثقة المستخدمين. المصادقةتعتمد معظم المنصات على اسم المستخدم وكلمة المرور فقط، مما يسهل اختراق الحسابات وانتحال الشخصية2غياب التوثيق القانوني للعقودالاتفاقيات تكون مجرد رسائل نصية غير موثقة، ويمكن لأي طرف التلاعب بها أو إنكارها3انتشار الحسابات المزيفةبسبب ضعف إجراءات التحقق من الهوية، تنتشر حسابات وهمية تغش المستخدمين

أهداف المشروع :

يهدف المشروع إلى بناء منصة متكاملة لتبادل المهارات تعتمد بنية الأمان المطلق ((Zero Trustعبر تحقيق الأهداف التالية: تطوير نظام مصادقة بيومترية متعدد العوامل يدعم كشف الحيوية لـ 4 وسائل تحقق، وتوثيق العقود والملفات زمنياً على شبكة Ethereum Sepolia و .IPFSكما يهدف لتطبيق التوقيع الرقمي بموجب خوارزمية RSA، وبناء نظام توصيات ذكي فائق السرعة عبر FAISS يتيح تفسير القرارات، وتوفير دردشة مشفرة بآلية .AES-256وأخيراً، حساب نظام ثقة ذاتي التعلم ((Trust Scoreمبني على 5 عوامل، مع إدارة شاملة عبر لوحة تحكم متكاملة ومحمية بأنظمة الحماية الموزعة وسجلات التدقيق.

نطاق المشروع :

تحدد نطاق المشروع بتوفير منصة عالمية لتبادل المهارات موجهة للمستقلين وأصحاب العمل، مع دعم كامل للغتين العربية والإنجليزية وتركيز خاص على المنطقة العربية. وتتيح المنصة منظومة خدمات متكاملة تبدأ من إدارة الملفات الشخصية ونشر طلبات العمل والبحث الذكي، مراراً بالدردشة الفورية المشفرة والتقييم المتبادل، وصولاً إلى تنظيم ثلاثة أنواع من العقود تشمل العقود ثابتة القيمة، والساعية، والمستندة إلى مراحل الإنجاز. وضماناً لأعلى مستويات الأمان، يقتصر النظام في التحقق البيومتري على خمس وسائل محددة هي الوجه، والصوت، وبصمة الإصبع، والتوقيع اليدوي، بالإضافة إلى رمز PIN الرقمي، في حين تُوثّق كافة العقود والتعاملات رسمياً عبر شبكة البلوكشين التجريبية Ethereum Sepolia

تنظيم المشروع :

تم تقسيم المشروع إلى ستة فصول رئيسية، يغطي كل فصل جانباً مهماً من جوانب النظام، وذلك على النحو التالي:

- **1. الإطار العام للمشروع :**

المحتويات الرئيسية: المقدمة، بيان المشكلة، أهداف المشروع، نطاق المشروع، تنظيم المشروع

- **2. دراسة التطبيقات المشابهة :**

المحتويات الرئيسية: تحليل معمق للمنصات المنافسة (Upwork، Fiverr، مستقل، خمسات، DocuSign، LinkedIn)، جدول مقارنة شامل، استخلاص نقاط القوة والضعف، تحديد الفجوات التي يسدها نظامنا.

- **3. الدراسة النظرية للتقنيات المستخدمة :**

المحتويات الرئيسية: شرح نظري مفصل لجميع التقنيات: التوقيع الرقمي ((RSA، التشفير ((SHA-256، البلوكشين والعقود الذكية((Ethereum Sepolia، المصادقة البيومترية (وجه، صوت، بصمة إصبع، توقيع)، نظام الصوت SpeechBrain V13نظام الوجه ((DeepFace + MTCNN، التوصيات (FAISS)، WebSocket، Django Channels، JWT، IPFS.

- **4. الدراسة التحليلية والتصميم :**

المحتويات الرئيسية: متطلبات النظام (وظيفية ولا وظيفية)، مخططات UML حالة الاستخدام، الصفوف، انتقال الحالة، التسلسل)، تصميم قاعدة البيانات (ERD، جداول، علاقات)، بنية النظامArchitecture Diagram مخطط انسيابية البيانات

- **5. التنفيذ والتطبيق (واجهات المشروع) :**

شرح التدفق الكامل للعمليات (سيناريوهات الاستخدام)، عرض جميع واجهات النظام مع شرح مفصل.

- 6. التحديات والاقتراحات المستقبلية :

المحتويات الرئيسية: التحديات التي واجهت فريق العمل أثناء التنفيذ (تقنية، إدارية، زمنية)، النتائج المتحققة (اختبارات الأداء والأمان)، الخاتمة، التوصيات لتطوير النظام مستقبلاً، المراجع.

الفصل الثاني :

دراسة للتطبيقات المشابهة

Capter Tow :

Study Similer Applications

المقدمة:

في عصر التكنولوجيا المتقدمة، أصبحت التطبيقات جزءاً لا يتجزأ من حياتنا اليومية. ومع تزايد الاعتماد على الهواتف الذكية والأجهزة اللوحية، أصبح من الضروري للمستخدمين إيجاد التطبيقات التي تلبي احتياجاتهم بشكل فعال وسهل. في هذا السياق، يظهر تطبيقنا كأحد الحلول المبتكرة التي تسعى لتقديم تجربة فريدة للمستخدمين. ومع ذلك، لا يمكننا تجاهل وجود العديد من التطبيقات المشابهة التي تسعى لتحقيق أهداف مشابهة.

سنتناول في هذا الحديث مجموعة من التطبيقات المنافسة التي تقدم ميزات وخدمات مشابهة لتطبيقنا، سنستعرض نقاط القوة والضعف في كل تطبيق، وكيف يمكن أن نستفيد من هذه المعرفة لتحسين خدماتنا وتقديم تجربة مستخدم أفضل.

التطبيق الأول:

تطبيق لينكد إن (LinkedIn) :



الشكل (1-2) أيقونة تطبيق LinkedIn

تطبيق LinkedIn هو أكبر منصة شبكات مهنية في العالم، وهو مصمم خصيصاً لمساعدة المحترفين وأصحاب الأعمال على بناء علاقات مهنية، تبادل الخبرات، والبحث عن فرص عمل. يتيح التطبيق للمستخدمين إنشاء ملف شخصي يمثل سيرة ذاتية رقمية متكاملة تشمل المهارات، الخبرات، والشهادات الأكاديمية، مما يجعله وجهة أساسية للشركات للبحث عن الكفاءات وتوظيفها.

✓ **مميزات تطبيق لينكد إن (LinkedIn) :**

- بناء شبكة علاقات مهنية واسعة مع محترفين في نفس المجال عالمياً.
- إمكانية نشر المحتوى والمقالات لتعزيز الهوية الرقمية (.Personal Branding)
- نظام توثيق للمهارات والشهادات من خلال تزكيات الزملاء (Endorsements)
- توفير منصة تعليمية متكاملة (LinkedIn Learning) لتطوير المهارات.
- أدوات بحث متقدمة للوظائف وتصفية الشركات بناءً على الموقع والتخصص.

✓ **سلبيات تطبيق لينكد إن (LinkedIn) :**

- لا يحتوي على نظام مدمج لإدارة وتوثيق العقود القانونية بين الأطراف.
- يفتقر إلى آليات التحقق البيومتري المتقدمة لحماية الهوية ومنع الحسابات الوهمية.
- كثرة المحتوى غير المهني أو الإعلاني في الآونة الأخيرة مما يشتت المستخدم.

✓ **لغات البرمجة المستخدمة في تطوير تطبيق لينكد إن (LinkedIn) :**

يعتمد تطبيق LinkedIn على بنية تحتية ضخمة وموزعة تستخدم مجموعة متنوعة من لغات البرمجة والتقنيات لتوفير الأداء العالي:

- iOS تُستخدم لغة Swift و Objective-C لبناء التطبيق ل أجهزة آبل.
- Android تُستخدم لغة Java و Kotlin لتطوير التطبيق الموجه لأنظمة أندرويد.
- تطوير الواجهة الأمامية:(Front-End)
- JavaScript / TypeScript للاعتماد على إطار العمل React لبناء واجهات مستخدم ديناميكية وسريعة الاستجابة.
- HTML5 / CSS3 لإنشاء الهيكلية البنائية والتصاميم المتوافقة مع كافة الشاشات.
- تطوير الواجهة الخلفية:(Back-End)
- Java اللغة الأساسية للخدمات الخلفية وأنظمة التخزين وإدارة البيانات الضخمة.
- Node.js تُستخدم كطبقة وسيطة (Middleware) لتسريع معالجة الطلبات اللحظية.
- Python / Go لبناء المصغرات البرمجية (Microservices) وأنظمة تحليل البيانات والتوصيات.

التطبيق الثاني :

تطبيق أب ورك (Upwork) :



الشكل (2-2) أيقونة تطبيق Upwork

تطبيق Upwork هو أحد أشهر المنصات العالمية الرائدة في مجال العمل الحر ((Freelancing))، حيث يربط بين أصحاب المشاريع والشركات من جهة، وبين المستقلين ذوي المهارات المتنوعة من جهة أخرى. يوفر التطبيق بيئة عمل رقمية متكاملة لإدارة المشاريع بدءاً من تقديم العروض، مروراً بالمحادثات الفورية، وصولاً إلى توقيع العقود وإتمام المدفوعات بشكل آمن.

✓ **مميزات تطبيق أب ورك (Upwork) :**

- نظام مالي آمن (Escrow)يضمن حقوق المستقل وصاحب العمل عند إتمام المراحل.
- دعم عدة أنواع من العقود (ذات السعر الثابت، أو العقود بالساعة عبر تعقب الوقت).
- نظام تقييم دقيق يعتمد على مؤشر نجاح المشاريع (Job Success Score)
- أدوات متكاملة لإدارة المشاريع ومشاركة الملفات والدردشة الفورية.

✓ **سلبيات تطبيق أب ورك (Upwork) :**

- اقتطاع عمولات مرتفعة جداً من أرباح المستقلين والمشترين.
- عمليات التحقق من الهوية تقليدية (عبر وثائق حكومية ومكالمات فيديو) وليست لحظية أو بيومترية مدمجة.

- لا يعتمد على تقنيات البلوكشين في توثيق العقود، مما يجعله نظاماً مركزياً بالكامل.

✓ **لغات البرمجة المستخدمة في تطوير تطبيق أب ورك (Upwork) :**

تعتمد منصة Upwork على بنية برمجية قوية قادرة على تحمل ملايين العمليات المالية والتعاقدية:

- React Native: يُستخدم لتطوير أجزاء مشتركة من التطبيق لتسريع الأداء بين منصتي iOS وأندرويد، إلى جانب التطوير الأصيل (Native) باستخدام Swift و.Kotlin
- تطوير الواجهة الأمامية:(Front-End)
- JavaScript: بالاعتماد بشكل أساسي على إطارات عمل حديثة مثل Angular و React لضمان سلاسة لوحات التحكم وإدارة العقود.
- تطوير الواجهة الخلفية:(Back-End)
- Java (Spring Boot)البنية الأساسية لمعالجة المنطق البرمجي المعقد والأنظمة المالية اللامركزية داخل التطبيق.
- Python (Django / FastAPI) لبناء خوارزميات البحث الذكي وتوصية المشاريع وحساب نسب نجاح المستقلين.
- PHP يُستخدم في بعض الأجزاء والأنظمة القديمة المرتبطة بإدارة المحتوى والصفحات الأساسية

التطبيق الثالث :

تطبيق فايفر (Fiverr) :



الشكل (3-2) أيقونة تطبيق Fiverr

تطبيق Fiverr هو أحد أبرز المنصات العالمية الرائدة في سوق العمل الحر الرقمي، ويقوم على مفهوم مبتكر يعتمد على تحويل المهارات إلى "منتجات جاهزة للشراء" تُعرف بالـ (.(Gigsيربط التطبيق بين المستقلين (البائعين) والعملاء (المشترين) من خلال سوق رقمي يتيح تصفح الخدمات بأسعار تبدأ غالباً من 5 دولارات وتتدرج بناءً على حجم وتفاصيل الخدمة المطلوبة، مع توفير نظام متكامل للمحادثات وإدارة الطلبات.

✓ مميزات تطبيق فايفر (Fiverr) :

- سهولة وسرعة التعاقد: لا يحتاج صاحب العمل لنشر مشروع وانتظار العروض، بل يمكنه شراء الخدمة الجاهزة مباشرة بنقرة واحدة.
- نظام حماية مالي: يعمل كوسيط مالي يضمن حفظ أموال المشتري وعدم تحريرها للبائع إلا بعد استلام الخدمة والموافقة عليها.
- تنوع حزم الخدمات: يتيح للمستقل تقديم ثلاث حزم مختلفة (أساسية، متوسطة، ومتقدمة) لنفس المهارة لتلبية ميزانيات العملاء المختلفة.
- بنية محادثات منظمة: يوفر أدوات دردشة مدمجة تتيح تبادل الملحقات، العينات، والاتفاق على التخصيصات الإضافية للطلب.

✓ سلبيات تطبيق فايفر (Fiverr) :

- عمولات اقتطاع مرتفعة: يقتطع التطبيق نسبة تصل إلى 20% من أرباح المستقل، بالإضافة إلى رسوم إضافية تفرض على المشترين.
- آليات التحقق من الهوية: يعتمد على طرق مركزية وتقليدية بطيئة للتحقق من الحسابات (مثل رفع جواز السفر أو الهوية الوطنية اليدوي) ويفتقر للتحقق البيومتري الفوري وكشف الحيوية.
- نظام مركزي بالكامل: تُخزن جميع العقود والبيانات والملفات المتبادلة في خوادم مركزية تابعة للشركة، مما يجعلها عرضة للتحكم الفردي أو الاختراق، على عكس أنظمة البلوكشين والـ .IPFS

✓ لغات البرمجة والتقنيات المستخدمة في تطوير تطبيق فايفر (Fiverr) :

تعتمد منصة Fiverr على معمارية برمجية هجينة وموزعة لضمان معالجة مئات آلاف الطلبات المتزامنة

- React Native يُعتمد عليه بشكل كبير لبناء وتوحيد واجهات التطبيق على منصتي iOS وأندرويد مع الحفاظ على الأداء العالي وسرعة التحديث.
- تطوير الواجهة الأمامية:(Front-End)

- React.js & TypeScript لإنشاء واجهات مستخدم تفاعلية وسلسة جداً، مخصصة لعرض الخدمات وتصفح الحزم بشكل ديناميكي سريع.
- تطوير الواجهة الخلفية:(Back-End)
- Ruby on Rails الإطار الأساسي والتاريخي للبنية الخلفية لفايفر، ويُستخدم لإدارة منطق العمل التجاري وعمليات الشراء والطلبات بشكل سريع وسلس.
- Python تُستخدم بشكل أساسي لبناء أنظمة التوصية الذكية(Recommendation Engines) ، وتحليل سلوك المستخدمين لاقتراح الخدمات المناسبة لاهتماماتهم.
- Node.js يُستفاد منه في بناء الخدمات المصغرة (Microservices) التي تتطلب معالجة فورية وتزامناً عالياً مثل نظام الإشعارات والمحادثات الآنية

المقارنة :

مقارنة مرجعية وتحليلية بين الأنظمة المشابهة والمشروع المقترح:

بعد استعراض ودراسة أهم المنصات والتطبيقات العالمية الرائدة في مجالات التواصل المهني والعمل الحر، وللوقوف على الثغرات الأمنية والتعاقدية والمالية التي تعاني منها هذه الأنظمة المركزية، تم إعداد دراسة مقارنة مرجعية (.(Benchmark Analysisيهدف هذا التحليل إلى إبراز مواطن القوة والابتكار الهندسي في مشروعنا المقترح، وتوضيح كيف يساهم دمج تقنيات الذكاء الاصطناعي والبلوكشين في تقديم بيئة تعاقدية آمنة وموثوقة بالكامل.

يوضح الجدول (2-1) مقارنة تفصيلية بين التطبيقات المدروسة وممشروعنا من حيث نوع المنصة، التقنيات المستخدمة، النقاط الأساسية، والفرق الجوهري الذي يثبت تفوق نظامنا

التطبيق	نوعه	اللغة	نقاط أساسية	الفرق بينه وبين مشروعنا
تطبيق لينكد إن (LinkedIn)	منصة شبكات وتواصل مهني وتوظيف عالمية.	يدعم اللغة العربية ولغات أخرى، ويعتمد على React و JavaScript في واجهاته.	يركز على بناء شبكات العلاقات المهنية. يعتمد على التزكيات التقليدية للمهارات من الزملاء. يمثل سيرة ذاتية رقمية متكاملة للمستخدمين.	قصور تعاقدي كامل . ضعف المصادقة والأمن يسهل انتشار الحسابات الوهمية.
تطبيق أب ورك (Upwork)	منصة عالمية وسيطة للعمل الحر.	يدعم اللغة الإنجليزية بشكل أساسي، ويعتمد على Angular و React في واجهاته.	يعتمد على تقديم العروض المكتوبة على المشاريع المطروحة. يوفر نظاماً مالياً وسيطاً يقيم المستقلين عبر مؤشر نجاح المشاريع الرقمي	المركزية هو نظام مركزي بالكامل. عمولات ضخمة بحث تقليدي .
تطبيق فايفر (Fiverr)	سوق رقمي عالمي لبيع الخدمات المصغرة الجاهزة	يدعم لغات عالمية متعددة، ويعتمد على React و TypeScript في واجهاته.	يتيح للمستخدمين شراء الخدمات الجاهزة. يوفر نظام حماية مالي وسيط بين البائع والمشتري. يتيح تقديم حزم خدمات متنوعة تلائم الميزانيات المختلفة.	محادثات بطيئة وتوثيق هوية يدوي بطيء

الجدول (2-1) مقارنة تفصيلية بين التطبيقات المدروسة

الفصل الثالث :

الدراسة النظرية للمشروع

Chapter Three :

Theoretical Study Of The Project

المقدمة :

يمثل الجانب النظري لأي نظام برمجي متقدم الركيزة الأساسية التي تستند إليها عمليات التحليل، والتصميم، والتنفيذ الفعلي. ومع تسارع وتيرة التحول الرقمي وظهور منصات العمل الحر وتبادل المهارات، برزت تحديات جوهرية تتعلق بضمان موثوقية الهوية الرقمية للمتعاملين، واستقرار القيمة القانونية للتعاقدات الإلكترونية، وحماية البيانات المتبادلة من التلاعب أو الإنكار. يأتي هذا الفصل لتقديم دراسة نظرية معمقة للمفاهيم العلمية والخوارزميات الهندسية يستعرض الفصل الأساس الرياضي والتقني لمنظومة الأمان الهجينة التي تجمع بين تشفير المفتاح العام (RSA) وتابع البصمة الرقمية(SHA-256) ، صعوداً إلى دراسة معمارية تقنية البلوكشين (Blockchain) والعقود الذكية عبر شبكة Ethereum\ Sepolia كأداة لتحقيق اللامركزية والشفافية وعدم قابلية التعديلكذلك، يغوص هذا الفصل في الجانب العلمي لتطبيقات الذكاء الاصطناعي والتعلم العميق المدمجة في النظام؛ حيث يتم تشريح خوارزميات معالجة الإشارة الصوتية ونموذج الشبكة العصبية المتقدم (ECAPA-TDNN) المستخدم لاستخراج البصمات الصوتية الحيوية، بجانب تقنيات الرؤية الحاسوبية وكشف الحيوية للوجوه (Liveness\ Detection) لمنع الاختراقات والتزييف. كما يتم التطرق إلى المفهوم الهندسي للبحث عن المتجهات عالي السرعة باستخدام مكتبة FAISS لبناء محرك التوصيات الذكي. وفي الختام، يوضح الفصل البرتوكولات الهندسية المستخدمة في إدارة الاتصال اللحظي ثنائي الاتجاه عبر الـWebSockets ، لينتهي بالتعريف ببيئة التطوير والأدوات البرمجية المعتمدة في بناء الواجهتين الأمامية والخلفية (React.js) و (Django\ REST\ Framework)وآلية التكامل والربط الفعال بينهما لضمان أداء مستقر، آمن، وفائق السرعة .

❖ المحور الأول: البيئة اللامركزية وبنية التشفير والحماية :

➢ تقنية البلوكشين (Blockchain) واللامركزية :

تُعرّف تقنية البلوكشين بأنها دفتر أستاذ موزع ومقاوم للتلاعب، حيث تُخزن البيانات على شكل سلسلة من الكتل الرقمية (Blocks) المرتبطة ببعضها البعض زمنياً وجغرافياً عبر شبكة من العقد (Nodes) دون الحاجة إلى سلطة مركزية أو وسيط ثالث موثوق.

يعتمد النظام على الخصائص الهندسية التالية للبلوكشين:

1. اللامركزية :(Decentralization) يتم توزيع سجلات العقود والاتفاقيات بين أطراف منصة تبادل المهارات على كامل الشبكة، مما يلغي نقطة الفشل الفردية (Single\ Point\ of\ Failure)

2. **الشفافية وقابلية التدقيق (Auditability):** يمكن لأي طرف ثالث أو جهة تحكيم مستقلة الاطلاع على السجل العام للتأكد من سلامة العقد ومطابقته للشروط الأصلية المتفق عليها.

3. **عدم قابلية التعديل (Immutability):** بمجرد كتابة بيانات العقد وتوقيعه على البلوكشين، يصبح من المستحيل رياضياً وبرمجياً تعديل تلك البيانات أو حذفها، مما يضمن بيئة عمل صارمة وموثوقة.

- **شبكة الإيثيريوم الاختبارية(Ethereum Sepolia) :**

من أجل تفعيل العقود الذكية بشكل حقيقي دون تكبد تكاليف مالية حقيقية أثناء مرحلة التطوير والاختبار، يعتمد مشروعنا على شبكة **Sepolia**، وهي شبكة الاختبار (Testnet) الرسمية المعتمدة لتطوير تطبيقات الإيثيريوم الحية وتجريبها ببيئة تحاكي الشبكة الرئيسية (Mainnet) تماماً.

***آلية الإجماع (Consensus\ Mechanism):** تعمل شبكة Sepolia وفق آلية إجماع إثبات الحصة (Proof\ of\ Stake - PoS)، مما يضمن سرعة إنتاج الكتل وثبات زمن المعاملات وجعلها مثالية لمنصتنا عند تسجيل الاتفاقيات وتأكيدها بشكل فوري.

- **رسوم الغاز (Gas Fees) وآلية عملها :**

تُمثّل "رسوم الغاز "وحدة القياس الداخلية المُستخدمة لتقدير حجم الجهد الحسابي والعمليات الرياضية المطلوبة لتنفيذ أمر برمجـي أو معالجة معاملة ما على شبكة الإيثيريوم.

حساب التكلفة :يتم احتساب تكلفة توثيق العقود وتثبيتها عبر المعادلة الرياضية التالية:

$$\{\text{Total Gas Fee}\} = \{\text{Gas Limit}\} \times \{\text{Base Fee}\}$$

وحيث أن نظامنا يهدف إلى تحسين الأداء وتقليل التكلفة، فقد تم بناء البنية التحتية البرمجية لتستدعي العقود الذكية بكفاءة عالية تختصر العمليات الحسابية المخزنة على الكتل لتوفير استهلاك الغاز، والاعتماد على سجل الأحداث الداخلي الموازي للمعاملات الحقيقية.(BlockchainAuditLog)

- **العقود الذكية(Smart Contracts) :**

العقود الذكية هي بروتوكولات حاسوبية ذاتية التنفيذ (Self-Executing\ Scripts)مخزنة مباشرة على البلوكشين، وتحتوي على شروط الاتفاقية البرمجية بين الطرفين (مثل: طالب الخدمة ومقدّم المهارة) بشكل مشفّر وصارم.

دورها في المشروع : بمجرد تحقق المنصة من الهوية الحيوية خماسية العوامل للطرفين وتوقيع العقد، يقوم العقد الذكي بربط الهوية الرقمية الفريدة للمستخدم بالمعاملة، وإصدار طابع زمني عالمي موثق وغير قابل للتلاعب عبر (OpenTimestamps) مما يضمن تحقيق مبدأ عدم قابلية الإنكار (Non-repudiation) فلا يمكن لأي مستخدم إنكار توقيعه أو تراجعه عن شروط العقد بعد تسجيله على السجل العام (Ledger)

➢ **.دفتر تسجيل الأحداث الموجّه بالأحداث(Event-Driven Ledger)**

بما أن التعامل المباشر المستمر مع شبكة البلوكشين (مثل الاستعلام عن حالات العقود أو جلب تفاصيل المعاملات في كل ثانية) يتسبب في بطء استجابة النظام وضياع الموارد، يعتمد مشروعنا على بنية هجينة متطورة تُعرف بـ **البنية الموجّهة بالأحداث.(Event-Driven Architecture)**

يتجسد هذا المفهوم برمجياً عبر نظام السجل الداخلي الموازي BlockchainAuditLog و BlockchainBlockبمجرد إتمام التوقيع الرقمي وإرسال المعاملة إلى شبكة Sepolia الحقيقية، يقوم النظام بالإنصات لحدث التأكيد الصادر من الشبكة(Blockchain Event) ، وفور التقاطه، يتم تسجيل "نسخة مطابقة "وموثقة من الـ Hash والطابع الزمني والبيانات الأساسية محلياً في دفتر أحداث داخلي خاضع للتدقيق. تضمن هذه الآلية الهندسية تقديم استجابة فائقة السرعة للمستخدم عند تصفح عقوده، مع الحفاظ على الأمان المطلق وربط البيانات بمرجعيتها الحقيقية غير القابلة للتعديل على البلوكشين.

➢ **نظام تشفير المفتاح العام والخاص (RSA Algorithm) :**

يمثل نظام التشفير غير المتناظر (Asymmetric\ Cryptography) باستخدام خوارزمية RSA الحجر الأساس لتوليد التواقيع الرقمية الطرفية للمستخدمين داخل المنصة قبل إرسالها إلى البلوكشين.

1. **آلية توليد المفاتيح (Key Generation) :**

*يعتمد على اختيار عددين أوليين كبيرين جداً وسريين (p) و.(q)

*يتم حساب المودولوس.(n = p \times q)

*يتم حساب دالة أيلر.((\phi(n) = (p-1) \times (q-1))

*يتم اختيار مفتاح التشفير العام (e) بحيث يكون أولياً مع.\phi(n)

*يتم حساب مفتاح فك التشفير الخاص (d) بحيث يحقق المعادلة:

D * E = 1 (mod phi(n))

2. الاستخدام في المنصة للتوثيق والتوقيع:

* المفتاح العام (Public\ Key) يُخزن في قاعدة بيانات النظام المرتبطة بملف المستخدم ويتاح للجميع للتحقق من صحة التوقيع.

* المفتاح الخاص (Private\ Key) يظل تحت الحماية الكاملة للمستخدم ويُستدعى لتوليد التوقيع الرقمي للوثائق والعقود. عند رغبة المستخدم بتوقيع عقد، يتم تشفير ملخص العقد بمفتاحه الخاص لينتج التوقيع الرقمي الفريد.

➢ **. تابع البصمة الإلكترونية ((SHA-256 Hash Function))**

لضمان استحالة التلاعب بمحتوى العقود أو ملفات المشاريع والاتفاقيات، يتم تمرير أي وثيقة أو نص عقد عبر تابع الـ Hash العالمي SHA-256 تابع التجزئة الآمن ذو الـ 256 بت .

*** خصائص الهندسة الأمنية لـ SHA-256 في المشروع:**

* أحادية الاتجاه (One-Way\ Function) يمكن بسهولة تحويل نص العقد إلى بصمة رقمية بطول ثابت يبلغ 64 محرفاً ست عشرّياً (256 بت)، ولكن من المستحيل رياضياً استرجاع النص الأصلي للعقد من خلال هذه البصمة.

* مقاومة التصادم (Collision\ Resistance) يضمن التابع عدم وجود وثيقتين أو عقدين مختلفين ينتجان نفس البصمة الرقمية على الإطلاق.

* تأثير الانهيار الجليدي (Avalanche\ Effect) لو تم تغيير حرف واحد أو فاصلة واحدة في شروط العقد، ستتغير البصمة الرقمية الناتجة بالكامل، مما يتيح لخادم Django والـ Smart Contract كشف أي محاولة تلاعب بالملفات فوراً وإبطال المعاملة.

❖ المحور الثاني: المصادقة البيومترية العصبية ونظام التوصيات:

➢ المصادقة متعددة العوامل (Multi-Factor Authentication - MFA) :

تعتمد الأنظمة الأمنية التقليدية على عوامل مصادقة تعتمد على المعرفة (مثل كلمات المرور) أو الحيازة (مثل بطاقات التعريف)، وهي وسائل معرضة للاختراق والسرقة. لذلك، يتجه مجمع هندسة البرمجيات إلى المصادقة الحيوية المدمجة (Multimodal Biometric Authentication)

في منصتنا، يتم تطبيق نظام مصادقة هجين يدمج خمسة عوامل أمنية (صوت، وجه، بصمة إصبع، رمز PIN، وتوقيع مشفر. الفكرة الهندسية هنا لا تعتمد على التحقق المنفرد، بل على مفهوم الاندماج الحسي ((Sensor Fusion على مستوى القرار Decision-Level Fusion حيث يعطي النظام خوارزمية ذكية تزن كل عامل حيوي بشكل منفصل، ولا تمنح صلاحية الوصول أو توقيع العقود إلا عند تخطي عتبة القبول الرياضية لكل عامل، مما يرفع دقة النظام ويخفض معدل القبول الخاطئ (FAR)إلى ما يقارب الصفر.

➢ معالجة الإشارة الصوتية ونموذج ECAPA-TDNN

تعتمد ميزة بصمة الصوت في المنصة على مكتبة SpeechBrain المتقدمة القائمة على الذكاء الاصطناعي. تمر الإشارة الصوتية الملتقطة من ميكروفون المستخدم بعدة مراحل هندسية ممتالية قبل عملية المطابقة:

1. مرحلة التقطيع والتنظيف (Preprocessing) : يتم تحويل الصوت التماثلي إلى إشارة رقمية بمعدل ترميز 16 HZ ، ثم تطبيق فلاتر لإزالة الضجيج المحيطي وعزل الصمت.

2. استخلاص الميزات (Feature Extraction) : يتم تحويل الإشارة الصوتية من النطاق الزمني إلى النطاق الترددي لحساب معاملات غابور السمعية المعروفة بـ MFCCs

(Mel-Frequency Cepstral Coefficients)

3. الشبكة العصبية ECAPA-TDNN: يتم تمرير هذه المعاملات إلى شبكة عصبية عميقة متقدمة تُعرف بـ * Emphasized Channel Attention, Propagation and Aggregation in Time Delay Neural Network*. تتميز هذه المعمارية بقدرتها على التركيز على القنوات الترددية الأكثر أهمية لصوت المتحدث عبر آليات الانتباه (Attention\ Mechanisms)

4. توليد المتجهات (Speaker Embeddings) : تقوم الشبكة باختزال المعطيات الصوتية المعقدة وتحويلها إلى مصفوفة رقمية أو متجه ثابت الطول (يُمثّل البصمة الصوتية الفريدة للمستخدم).

- **خوارزمية تشابه الجيوب (Cosine Similarity) للتحقق الصوتي**

عندما يقوم المستخدم بمحاولة تسجيل الدخول أو توقيع عقد، يقوم النظام باستخراج متجه الصوت الجديد (A) ومقارنته بالمتجه المخزن مسبقاً في قاعدة البيانات أثناء مرحلة التسجيل .(B) تعتمد عملية المقارنة الرياضية على **خوارزمية تشابه الجيوب (Cosine Similarity)** عبر المعادلة:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

***عتبة القبول الهندسي (Threshold):** يقوم الخادم في الملف المصدري (biometric/views.py) بحساب النتيجة؛ فإذا كانت قيمة التشابه أكبر من (0.5 <) **0.5** يتم قبول البصمة الصوتية كبصمة مطابقة وهوية موثوقة، وإلا يتم رفض المعاملة فوراً.

- **بصمة الوجه وكشف الحيوية (MTCNN + DeepFace)) :**

لمنع هجمات التزييف عبر الصور أو الفيديو المطبوع، يدمج النظام بين كشف الوجه والتحقق من الهوية وكشف الحيوية:(Liveness\ Detection)

1. **خوارزمية MTCNN (Multi-task Cascaded Convolutional Networks):** هي شبكة عصبية تلافيفية متتالية ثلاثية المراحل. تقوم برصد وجود الوجه في الصورة بدقة عالية، وتحديد النقاط الخمس الأساسية للمعالم (العينين، الأنف، وزاويتي الفم)، مما يسمح بعمل محاذاة وتدوير تلقائي للوجه لتصحيح زاوية التصوير قبل المعالجة.

2. **إطار العمل DeepFace :** يتم تمرير الوجه المحاذى إلى نموذج DeepFace المستند إلى معمارية FaceNetأو VGG-Face لاستخلاص متجه الميزات الرياضية للوجه.(Face\ Embeddings)

3. **آلية كشف التزييف والحيوية (Anti-spoofing / Liveness)** يختبر النظام انعكاس الضوء، وتحركات

العين الرمشية، والتغيرات الطفيفة في الملامح للتأكد من أن الوجه الذي يقف أمام الكاميرا هو وجه بشري حي وثلاثي الأبعاد وليس مجرد صورة فوتوغرافية معروضة أمام الهاتف أو الحاسب.

- **المصادقة عبر بصمة الإصبع والعتاد المدمج ((Suprema & Windows Hello)) :**

تكتمل منظومة التحقق البيومتري الطرفي في المنصة عبر دمج عامل بصمة الإصبع، حيث يتعامل النظام مع مستويات مختلفة من العتاد لضمان أعلى معايير الأمان المادي:

1. **تقنيات Suprema الإحترافية :**يعتمد النظام على المعايير الهندسية لأجهزة ومستشعرات شركة *Suprema*الرائدة، والتي توفر خوارزميات متقدمة لكشف البصمات الحيوية الزائفة (Live Finger Detection - LFD)، حيث يتم تحليل الخصائص الفيزيائية للإصبع (مثل النفاذية الضوئية والكهربائية) لمنع الاختراق باستخدام البصمات السيليكونية أو المطاطية.

2.**إطار عمل Windows Hello:** لضمان توافقية النظام وسهولة استخدامه على الحواسب الشخصية، يتم التكامل مع واجهات برمجة التطبيقات الخاصة بـWindows Hello*

.*(Biometric Framework) تتيح هذه البنية البرمجية للنظام استدعاء مستشعرات البصمة المدمجة في الأجهزة بشكل آمن تماماً، حيث يتم التحقق من الهوية محلياً على مستوى العتاد المحمي، ولا يتم تمرير سوى توكن تأكيد النجاح المشفر إلى خادمDjango ، مما يحافظ على خصوصية بيانات المستخدم الحيوية الخام ويمنع تسريبها عبر الشبكة.

- **محرك التوصيات والبحث عن المتجهات عبر مكتبة FAISS :**

عندما يبحث طالب خدمة عن مقدم مهارة (أو العكس)، فإن البحث النصي التقليدي عبر الكلمات المفتاحية ((Keyword\ Search)) يعتبر قاصراً عن فهم السياق الدلالي للمهارات. لذلك يعتمد مشروعنا على البحث الشعاعي الدلالي (Vector Search) باستخدام مكتبة FAISS (Facebook AI Similarity Search) المطورة من قِبل شركة .Meta .

1. **تحويل النص إلى متجهات (Text Embedding)** يتم تحويل النصوص الخاصة بالمهارات والخبرات والمشاريع إلى متجهات رقمية في فضاء متعدد الأبعاد يعكس المعنى الدلالي للمهارة.

2. **فهرسة البيانات (Indexing)** تستخدم المكتبة فهارس متقدمة مثل IndexFlatIP أو IVF (Inverted File Index) لتقسيم الفضاء الشعاعي إلى عناقيد (Clusters)

سرعة البحث وعملية التوصية: بدلاً من قيام النظام بعمل مسح كامل وقراءة كافة السجلات في قاعدة البيانات (والتي تأخذ زمناً خطياً \(mathcal{O}(N)\)، تقوم مكتبة FAISS بالبحث في العناقيد الأقرب فقط، مما يختصر زمن الاستجابة إلى زمن لوغاريتمي \ .(mathcal{O}(\log N يتيح ذلك للنظام

حساب المسافات الرياضية وترشيح أفضل المطورين أو المشاريع المتوافقة مع رغبات المستخدمين بدقة فائقة ولحظية، مضافاً إليها تفسيرات ذكية تُبرز سبب التوصية (Explainable\ AI)

- **خوارزمية حساب معدل الثقة الذكي ((Trust Score Algorithm)) :**

لا تقتصر عملية ترشيح المطورين وأصحاب المشاريع في المنصة على التطابق الدلالي للمهارات فحسب، بل تخضع لتقييم أمني وعملي صارم يُدعى **معدل الثقة** .**(Trust Score)** وهي خوارزمية رياضية ذكية متعددة العوامل (Multi-Criteria Decision Making) تعمل في الخلفية لتصنيف المستخدمين بناءً على سلوكهم الحقيقي داخل المنصة.

***العوامل الرياضية المحددة للمعدل :**يتم احتساب هذا السجل الديناميكي بناءً على الأوزان التالية:

1. **معدل الالتزام بالشروط :(Commitment Rate)** يمثل نسبة العقود المكتملة بنجاح دون إخلال أو لجوء للتحكيم.
2. **سرعة الإنجاز والتسليم :(Delivery Punctuality)** قياس مدى الالتزام بالطوابع الزمنية الموثقة على البلوكشين.
3. **تاريخ التقييمات :(Historical Rating)** المتوسط المرجح لتقييمات العملاء السابقين.
4. **استقرار الهوية الحيوية :(Biometric Consistency)** مدى نجاح المستخدم في تخطي اختبارات المصادقة الدورية دون محاولات مشبوهة.

يتم دمج هذه العوامل في معادلة تفاضلية تعطي مؤشراً رقمياً نهائياً للثقة يظهر بجانب ملف المستخدم، ويتم تمريره كعامل تصفية أساسي داخل محرك البحثFAISS ، مما يضمن إبراز المطورين الأكثر أماناً والملتزمين قانونياً، وعزل أي حسابات تبدي سلوكاً احتيالياً أو غير مستقر.

❖ المحور الثالث: بروتوكولات الاتصال وبيئة التطوير (الربط والبرمجة):

➢ بروتوكولات الاتصال ونقل البيانات اللحظية :

تعتمد تطبيقات الويب التقليدية على بروتوكول **HTTP**، وهو بروتوكول يعتمد على نموذج الطلب والاستجابة(Request-Response) ؛ حيث يرسل العميل (Client) طلباً ويقوم الخادم (Server) بالرد عليه ثم تُغلق القناة. لضمان تحديث البيانات، تُستخدم تقنيات مثل الـ Polling والتي تستهلك موارد الخادم بشكل كبير عبر إرسال طلبات متكررة دون فائدة.

لحل هذه المشكلة في منصتنا (خاصة في نظام الدردشة اللحظية والتحقق الفوري)، تم الاعتماد على بروتوكول .**WebSockets**

***آلية العمل :**يقوم البروتوكول بإنشاء اتصال مستمر، ثنائي الاتجاه(Full-Duplex) ، عبر فتح قناة اتصال واحدة عبر منفذ الـ .TCP تتيح هذه القناة تبادل الرسائل والبيانات (مثل حزم الصوت والرسائل

الفورية وتنبيهات العقود الذكية) بين الواجهة الأمامية والخلفية بلحظتها وبأقل زمن تأخير ممكن (Low\ Latency)دون الحاجة لإعادة إرسال ترويسات الـ HTTP في كل مرة.

تقنيات الواجهة الأمامية ((Frontend Environment)) :

تُبنى الواجهة الأمامية لنظام **skill swap**باستخدام مكتبة **React.js**مضافاً إليها التقنيات المعيارية لتصميم الويب، وذلك للأسباب الهندسية التالية:

1**مفهوم الواجهة الافتراضية :(Virtual\ DOM)** لا تقوم React بتحديث كامل الصفحة عند حدوث أي تغيير في البيانات (مثل تحديث حالة معالجة البصمة البيومترية)، بل تقوم بتحديث الأجزاء المتغيرة فقط ضمن الـ Virtual\ DOM وتطبيقها على المتصفح، مما يضمن أداءً فائق السرعة واستجابة سلسة.

2 **المكونات القابلة لإعادة الاستخدام :(Component-Based)** يتم تقسيم النظام إلى مكونات مستقلة مثل واجهة تسجيل البصمات (BiometricEnrollModal.jsx) ولوحة تحقق المصادقة (biometric modal .jsx)

3 . **الوصول إلى عتاد الجهاز عبر HTML5** يتم استغلال واجهات برمجة التطبيقات المتقدمة للمتصفح مثل navigator.mediaDevicesللوصول الآمن والمباشر إلى الكاميرا والميكروفون من أجل التقاط بصمات الوجه والصوت الخاصة بالمستخدم وإرسالها إلى الخادم.

تقنيات الواجهة الخلفية(Backend Environment) :

يمثل الخادم الخلفي النواة الحسابية والأمنية للمشروع، وتم بناؤه باستخدام لغة **Python**واعتماد إطار العمل **Django**ممتداً إلى بيئات متخصصة:

1.**إطار عمل Django REST Framework (DRF)** يُستخدم لبناء واجهات برمجة تطبيقات قوية وآمنة (RESTful\ APIs)تعتمد على صيغة JSON لتبادل البيانات. يدير الـ DRF عمليات التحقق من الصلاحيات وتوليد التوكنات الآمنة لجلسات المستخدمين عبر واجهات مثل.(TokenObtainPairView)

2 . **بيئة Django Channels** بما أن خادم Django التقليدي يعالج الطلبات المتزامنة (Synchronous) فقط عبر بروتوكولWSGI ، تم دمج Django\ Channels لتحويل الخادم ليعمل بآلية غير متزامنة (Asynchronous)عبر بروتوكول .ASGI تتيح هذه البيئة للخادم التعامل مع قنوات الـ WebSockets والموازنة بين معالجة طلبات الـ API التقليدية والاتصالات اللحظية المستمرة في نفس الوقت.

3. **تكامل معالجة البيانات والذكاء الاصطناعي :**تم اختيار لغة Python كخلفية برمجية نظراً لتوافقها المطلق والمباشر مع مكتبات معالجة البيانات الثقيلة والذكاء الاصطناعي المستدعاة داخل الأكواد) مثل SpeechBrain, DeepFaceوFAISS

.هندسة البيانات وقواعد البيانات الهجينة :

يدير النظام بنية بيانات هجينة تجمع بين السجلات العلائقية والتدقيق اللامركزي:

***قاعدة البيانات المحلية (SQLite / PostgreSQL)** تُستخدم لتخزين الجداول المنظمة مثل بيانات المستخدمين الأساسية، طلبات المهارات، وحالات العقود.

***سجل الأحداث الموازي للمستندات (Blockchain Audit Log)** لتجنب الاستعلام المستمر والمكلف لشبكة البلوكشين وتوفير "رسوم الغاز"، يقوم الخادم الخلفي بإدارة جدول موازٍ مخصص (BlockchainAuditLog)يقوم بتسجيل وحفظ الأحداث وهواش (Hashes) الكتل (BlockchainBlock)محلياً بشكل فوري بمجرد تأكيد المعاملة على شبكة Sepolia الحقيقية، مما يضمن سرعة التحقق وعرض البيانات للمستخدم مع الحفاظ على المرجعية المطلقة للبلوكشين.

الفصل الرابع:

دراسة تحليلية للمشروع

Chapter Four :

Analytical Study Of Project

مقدمة :

مخططات UML (Unified Modeling Language) هي أدوات تصويرية تُستخدم في هندسة البرمجيات لتصميم وتوثيق الأنظمة البرمجية. تُعتبر UML لغة موحدة لتصميم النماذج، تساعد على تمثيل الهيكلية والسلوك والتفاعلات داخل النظام بطريقة بصرية مفهومة. تُستخدم هذه المخططات من قبل المطورين والمحللين وأصحاب المصلحة لفهم النظام وتصميمه وتطويره.

وهنا سوف نضع المخططات التي استخدمناها في تصميم هيكلية مشروعنا.

مخطط حالة الاستخدام :

مخطط حالة الاستخدام (Use Case Diagram) هو أحد أنواع المخططات المستخدمة في نمذجة لغة النمذجة الموحدة (UML)، ويُستخدم بشكل رئيسي في تحليل وتصميم النظم البرمجية. يهدف إلى توضيح التفاعلات بين المستخدمين (أو الكيانات الخارجية، تُسمى Actors) والنظام لتحقيق أهداف محددة.

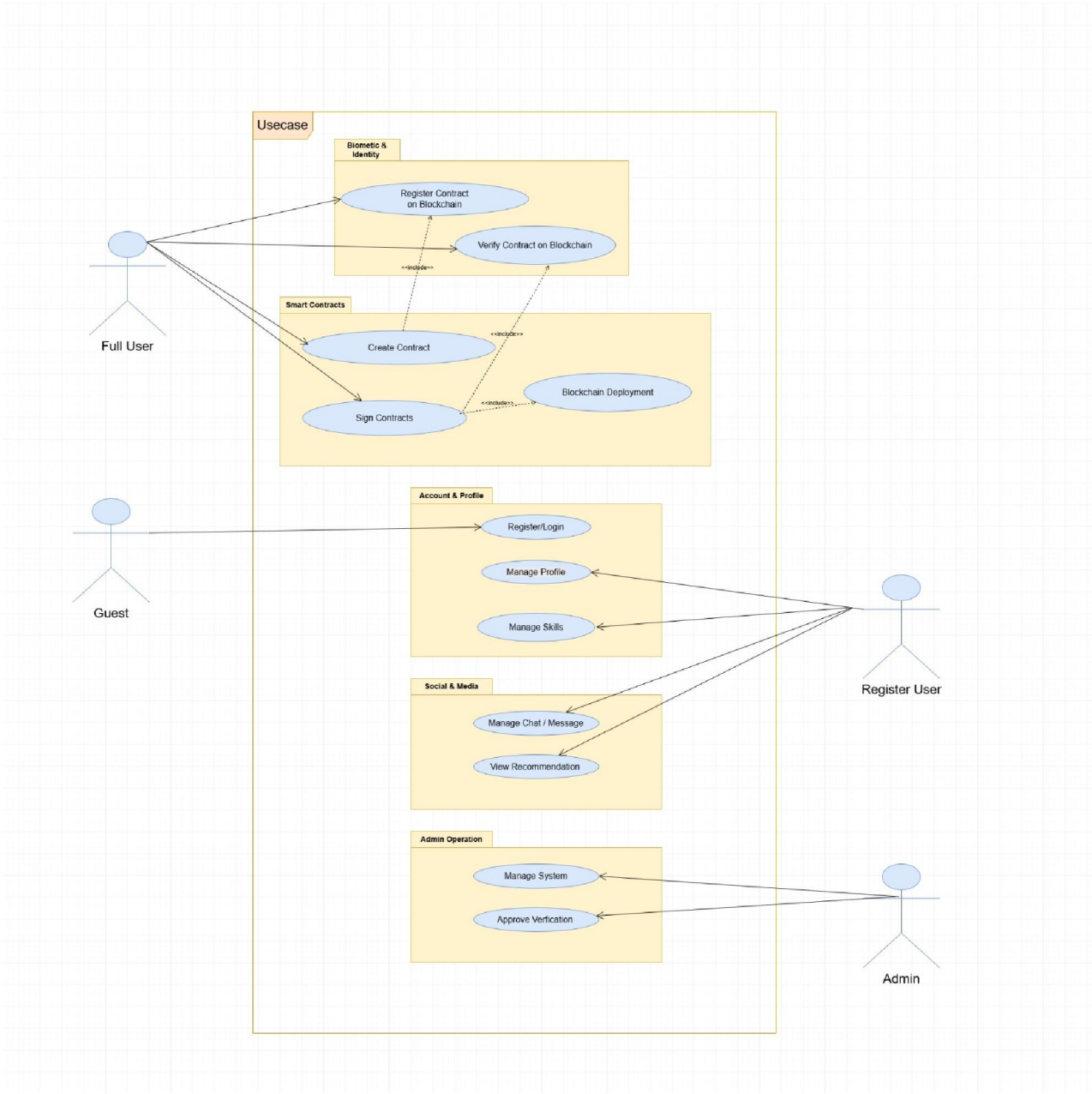
وظيفة مخطط حالة الاستخدام:

تحديد المتطلبات الوظيفية وتبسيط التواصل حيث يُستخدم كأداة تواصل بين فريق التطوير (المبرمجين والمحللين) والعملاء أو أصحاب المصلحة لضمان فهم مشترك للمتطلبات.

مكونات مخطط حالة الاستخدام:

يتكون من الفاعلين (Actors) وحالات الاستخدام (Use Case) والعلاقات التي تظهر الارتباطات بين الفاعل وحالة الاستخدام

والشكل الموضح أدناه يمثل مخطط حالة الاستخدام الخاص بموقعنا



الشكل (4-4) مخطط حالة الاستخدام

مكونات هذا المخطط :

1. **الفاعلون (Actors) :**

* **المستخدم الضيف (Guest)** : المستخدم غير المسجل في النظام، ويمتلك صلاحيات محدودة جداً مثل استعراض الواجهة الأساسية أو الدخول لإنشاء حساب جديد.

* **المستخدم المسجل (Register User)** :المستخدم الذي قام بإنشاء حساب وتأكيده، ويمتلك صلاحيات إدارة حساب الشخصي، وإدارة مهاراته، والدخول للدردشة واستعراض التوصيات.

* **المستخدم الكامل (Full User)** : المستخدم الذي أتمّ تفعيل خيارات الأمان والتحقق المتقدمة (البيومترية)، وله الصلاحية الكاملة في صياغة وتوقيع وتوثيق العقود على البلوكشين.

* **المشرف (Admin)** : مسؤول النظام الذي يشرف على العمليات الإدارية، ومراقبة أداء السيرفر، والموافقة على طلبات التحقق من الهوية.

2. **حالات الاستخدام (Use Cases) :**

حزمة إدارة الحساب والملف الشخصي(Account & Profile) :

التسجيل / تسجيل الدخول (Register/Login) :تتيح للمستخدم الضيف (Guest) إنشاء حساب جديد أو الدخول للنظام ليصبح مستخدماً مسجلاً.

*إدارة الملف الشخصي (Manage Profile) تتيح للمفوضين مثل (Register User)تعديل بياناتهم الشخصية، ورفع صورهم وصياغة النبذة الذاتية.

*إدارة المهارات (Manage Skills) تتيح للمستخدم المسجل إضافة، تعديل، أو حذف المهارات والخدمات التي يعرضها على المنصة.

حزمة التواصل والوسائط (Social & Media) :

* إدارة المحادثات والرسائل (Manage Chat / Message) : تتيح للمستخدمين المسجلين التواصل اللحظي وتبادل الرسائل.

* عرض التوصيات (View Recommendation) : تتيح للمستخدم المسجل استعراض المقترحات والتوصيات الذكية للمهارات بناءً على اهتماماته ونشاطه.

حزمة العقود الذكية (Smart Contracts) :

* إنشاء عقد (Create Contract) : تتيح للمستخدم الكامل (Full User) صياغة بنود العقد وتحديد تفاصيله وشروطه الأولية.

* توقيع العقود (Sign Contracts) :تتيح للمستخدم الكامل توقيع العقد رقمياً وبيومترياً للموافقة عليه والالتزام به.

* نشر المعاملة على البلوكشين (Blockchain Deployment) عملية معالجة داخلية خلفية لتثبيت العقد على الشبكة اللامركزية.

حزمة المؤشرات البيومترية والهوية (Biometric & Identity) :

* تسجيل العقد على البلوكشين (Register Contract on Blockchain) : ربط بيانات العقد وسجل التواقيع بالكتل اللامركزية بشكل دائم وثابت.

* التحقق من العقد على البلوكشين (Verify Contract on Blockchain) تمكين الأطراف من فحص وتدقيق صحة العقد وسلامته من أي تلاعب عبر الشبكة.

حزمة عمليات المشرف (Admin Operation) :

* إدارة النظام (Manage System) : تتيح للمشرف (Admin) مراقبة الأداء العام للمنصة، السيرفرات، وحل المشاكل التقنية.

* الموافقة على التحقق (Approve Verification) : تتيح للمشرف مراجعة وثائق الهوية المرفوعة وقبولها لترقية المستخدم إلى مستخدم كامل (Full User) .

3. العلاقات الخاصة (Relationships) :

* **علاقة التضمين (Include) :** تعني أن تنفيذ حالة الاستخدام الأساسية يتطلب ويستدعي إجبارياً تنفيذ حالة الاستخدام الأخرى كجزء لا يتجزأ من مسارها الإلزامي. وفي مخطط مشروعنا، تظهر العلاقات كالتالي:

* علاقة التضمين بين (Sign Contracts) و (Blockchain Deployment) :

عملية "توقيع العقود" من قبل الأطراف تتضمن تلقائياً وبشكل إلزامي <<include>> عملية "نشر المعاملة على البلوكشين" (Blockchain Deployment) حيث لا يمكن اكتمال التوقيع البيومتري والرقمي نهائياً دون تشغيل ونشر المعاملة على شبكة إيثيريوم لتثبيتها.

* علاقة التضمين بين (Register Contract on Blockchain) و (Create Contract) :

عملية "تسجيل العقد على البلوكشين" تتضمن تلقائياً وبشكل إلزامي <<include>> عملية "إنشاء عقد" (Create Contract) إذ لا يمكن لنظام البلوكشين تسجيل وثيقة أو عقد دون أن يمر بمسار البناء والصياغة الأولية لبنود الاتفاق أولاً.

* **علاقة الامتداد (Extend) :** تعني أن الوظيفة تعتبر ميزة إضافية أو مشروطة، ولا تُنفذ إلا في حالات معينة أو عند طلبها، ولا تؤثر على سير العملية الأساسية في حال عدم تنفيذها. وتظهر في مشروعنا كالتالي:

* علاقة الامتداد من (Create Contract) إلى (Verify Contract on Blockchain) :

حالة الاستخدام "التحقق من العقد على البلوكشين" يمكن أن تمتد وتُستدعى <<extend>> عند قيام المستخدم بـ "إنشاء عقد"؛ وذلك في حال رغب المنشئ بتدقيق ومقارنة العقد الحالي مع النسخ الموثقة مسبقاً على الشبكة للتأكد من سلامة المعاملات أو عدم تكرارها، وهي خطوة اختيارية تعتمد على رغبة المستخدم وسياق النظام.

ملخص المخطط:

يبين المخطط الأدوار والمسؤوليات المنفصلة حيث تبدأ رحلة المستخدم من كونه ضيفاً (Guest) ثم مستخدماً مسجلاً (Register User) قادراً على التواصل وإدارة مهاراته، وصولاً لترقيته كمستخدم كامل ((Full User مؤهل لإجراء المعاملات الحساسة والتواقيع الرقمية المرتبطة بـ البلوكشين والعقود الذكية، تحت إشراف وتدقيق كامل من مسؤول النظام (Admin) .

مخطط الصفوف :

مخطط الصفوف (**Class Diagram**) هو تمثيل بصري لتسلسل الخطوات أو العمليات في عملية أو نظام معين. يستخدم رموزًا موحدة مثل الأشكال الهندسية (مثل المستطيلات، المعينات، والدوائر) لتوضيح الخطوات، القرارات، وتدفق العمل.

وظيفة مخطط الصفوف:

توضيح العمليات و تحليل النظام و دعم حل المشكلات حيث يساعد في تحليل المشكلات وإيجاد حلول من خلال تصور العملية

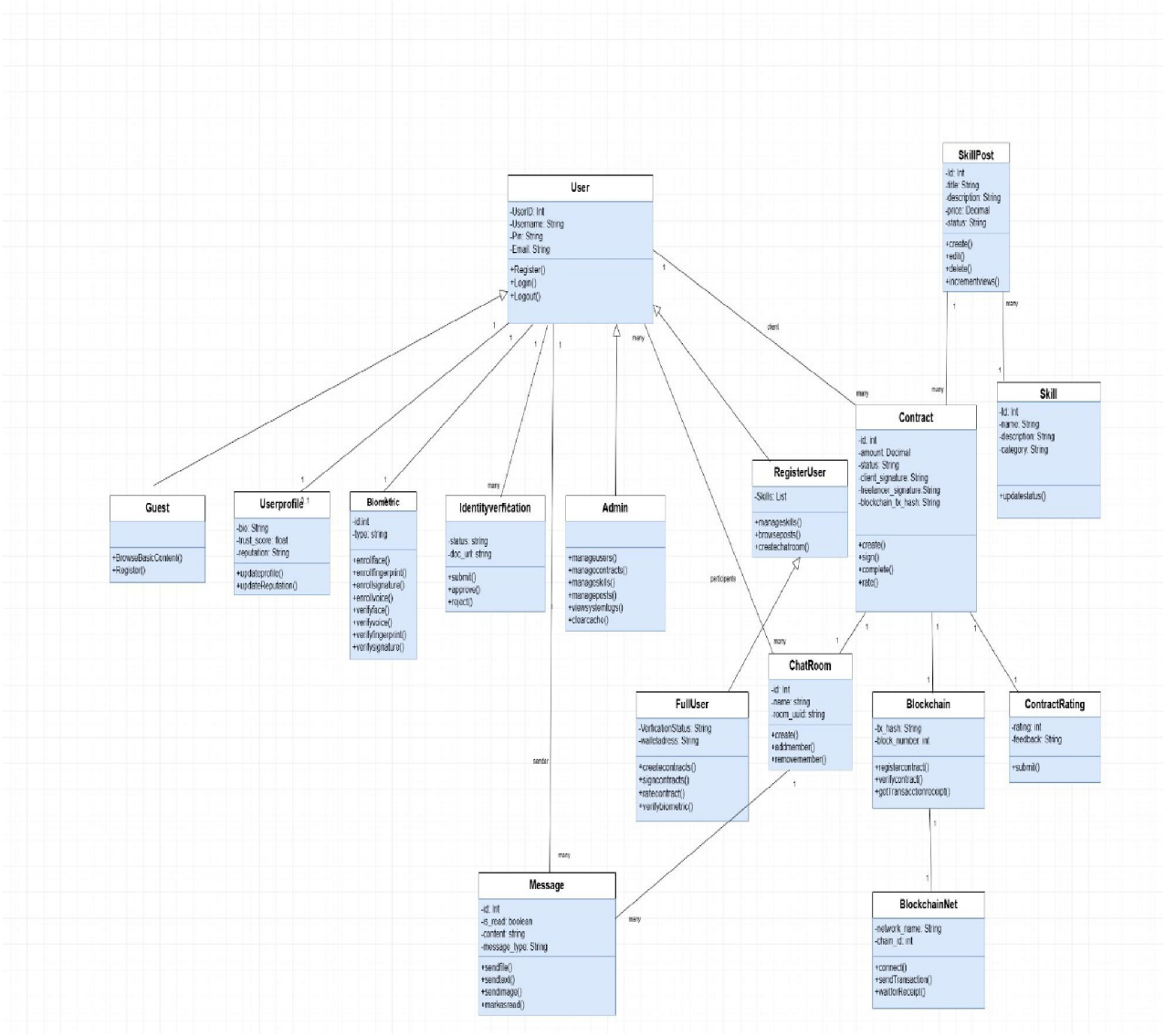
مكونات مخطط الصفوف الرئيسية:

الكيانات (**Entities**):تمثل الأشياء أو الكائنات في النظام التي لها خصائص

السمات (**Attributes**):تمثل خصائص أو صفات الكيان

العلاقات (**Relationships**):توضح كيفية ارتباط الكيانات ببعضها

والشكل الموضح أدناه يمثل مخطط الصفوف الخاص بموقعنا



الشكل (5-4) مخطط الصفوف

مكونات هذا المخطط هي:

* **المستخدم (User)** ؛ يحوي بيانات الحساب المشتركة والأساسية في النظام مثل:
  * UserID : المعرّف الرقمي الفريد للمستخدم.
  * Username : اسم المستخدم للحساب.
  * Pin : الرمز السري المكون من 8 أرقام كخيار أمان أولي.
  * Email : البريد الإلكتروني الخاص بالمستخدم.

*** الوظائف (Methods) :**

* Register : تسجيل مستخدم جديد في النظام.

* Login : تسجيل الدخول وحجم الصلاحيات.

* Logout : تسجيل الخروج وتدمير الجلسة الآمنة.

*** الملف الشخصي (UserProfile) : يحوي البيانات المهنية والتقييمية للمستخدم مثل:**

* bio : السيرة الذاتية والنبذة التعريفية للمستخدم.

* trust_score : مؤشر الثقة الرقمي المبني على الالتزام بالعقود.

* reputation : مستوى السمعة العامة المكتسبة داخل المنصة.

*** الوظائف (Methods) :**

* updateProfile : تحديث معلومات الملف الشخصي.

* updateReputation : تحديث وتعديل نقاط السمعة برمجياً.

*** البيانات الحيوية (Biometric) : مخصص لإدارة البيانات والمؤشرات البيومترية للمستخدم مثل:**

* id : المعرّف الرقمي الخاص بملف البصمات.

* type : نوع البصمة الحيوية الافتراضية المفعلة للجهاز.

*** الوظائف (Methods) :**

* enrollFace : تسجيل بصمة الوجه وحفظ الـ Embedding الخاص بها.

* enrollFingerprint : مسح وتسجيل بصمة الإصبع في النظام.

* enrollSignature : تسجيل نمط التوقيع الرقمي للمستخدم.

* enrollVoice : تسجيل البصمة الصوتية عبر المايكروفون.

* verifyFace : مطابقة وتدقيق بصمة الوجه الحالية.

* verifyVoice : مطابقة نموذج الصوت للتحقق التلقائي.

* verifyFingerprint : مطابقة بصمة الإصبع عند التوقيع.

* verifySignature : تدقيق ومصادقة التوقيع اليدوي الرقمي.

*** طلب التحقق من الهوية (IdentityVerification) : يخص وثائق معالجة ترقية الحسابات مثل:**

* status : حالة الطلب الحالي (قيد المراجعة، مقبول، مرفوض).

* doc_url : رابط أو مسار تخزين الوثيقة الرسمية المرفوعة.

*** الوظائف (Methods) :**

* submit : إرسال ورفع وثائق التحقق الرسمية للنظام.

* approve : موافقة المشرف وتغيير رتبة المستخدم.

* reject : رفض الطلب مع إرسال سبب الرفض.

*** مدير النظام (Admin) : يرث خصائص المستخدم ويحوي وظائف التحكم الإداري الشامل مثل:**

*** الوظائف (Methods) :**

* manageusers : التحكم والرقابة على حسابات المستخدمين.

* managecontracts : الإشراف على مسار العقود وحل النزاعات.

* manageskills : مراجعة وتدقيق المهارات المضافة بالمنصة.

* manageposts : التحكم بالمنشورات وتعديلها أو حذفها.

* viewsystemlogs : استعراض سجلات النظام والأحداث التقنية.

* clearcache : تنظيف وتفريغ الذاكرة المؤقتة لتحسين الأداء.

*** المستخدم المسجل (RegisterUser) : يرث الخصائص العامة ويمتلك وظائف المهارات والتواصل المبدئي مثل:**

* Skills : قائمة تحتوي على المهارات التي يبرع بها المستخدم.

*** الوظائف (Methods) :**

* manageskills : إضافة، تعديل، أو حذف المهارات الخاصة به.

* browseposts : تصفح المنشورات والبحث الذكي عن المهارات.

* createchatroom : إنشاء غرف محادثة جديدة للتفاوض.

*** المستخدم الكامل (FullUser) : يرث خصائص المسجل، ومخوّل بالعمليات المالية وعقود البلوكشين مثل:**

* verificationStatus : حالة التوثيق البيومتري المتقدم.

* walletaddress : عنوان المحفظة الرقمية اللامركزية الخاصة به.

*** الوظائف (Methods) :**

* createcontracts : صياغة وبدء إنشاء عقود مهارية جديدة.

* signcontracts : التوقيع والمصادقة النهائية على العقد.

* ratecontract : تقييم العقد والطرف الآخر بعد اكتمال العمل.

* verifybiometric : استدعاء واجهات التحقق الحيوي لتأكيد الهوية البيومترية.

*** منشور المهارة (SkillPost) : يحوي تفاصيل الإعلان والمهارة المعروضة للعمل مثل:**

* id : المعرّف الفريد للمنشور.

* title : عنوان منشور المهارة المعروضة.

* description : تفاصيل المنشور وشروطه.

* price : السعر المالي المقدر للخدمة أو المهارة.

*** الوظائف (Methods) :**

* create : نشر وإتاحة عرض مهارة جديد بالمنصة.

* edit : تعديل تفاصيل أو سعر المنشور الحالي.

* delete : حذف المنشور من محركات العرض.

* incrementviews : زيادة عدّاد المشاهدات والزيارات تلقائياً.

*** المهارة الفنية (Skill) : يحوي التفاصيل الهيكلية والتصنيفية للمهارات مثل:**

* id : معرّف المهارة البرمجي.

* name : اسم المهارة المحددة (مثال: تطوير ويب، تصميم، إلخ).

* description : وصف طبيعة ومتطلبات هذه المهارة.

* category : التصنيف الرئيسي الذي تنتمي إليه المهارة.

*** الوظائف (Methods) :**

* updatestatus : تحديث حالة المهارة داخل قواعد البيانات للنظام.

*** العقد الذكي اللامركزي (Contract) : يحوي بيانات ومعلومات الاتفاق القانوني والمالي المبرم بين الطرفين مثل:**

* id : المعرّف الرقمي الخاص بالوثيقة.

* amount : القيمة المالية والميزانية المخصصة للعقد.

* status : حالة العقد الحالية (مسودة، نشط، مكتمل، متنازع عليه).

* client_signature : سلسلة البيانات النصية لتوقيع العميل.

* freelancer_signature : سلسلة البيانات النصية لتوقيع المستقل.

* blockchain_tx_hash : معرّف المعاملة (Hash) الثابت على شبكة البلوكشين.

*** الوظائف (Methods) :**

* create : صياغة وبناء مسودة العقد الأساسية.

* sign : تجميع وحفظ التواقيع البيومترية والرقمية للأطراف.

* complete : إغلاق العقد وتحرير الدفعات بعد نجاح الفحص.

* rate : معالجة تقييم جودة الخدمة والالتزام بالبنود.

*** تقييم العقد (ContractRating) :يختص بحفظ تقييمات وملاحظات نهاية العمل مثل:**

* rating : عدد النجوم أو الدرجة الممنوحة

* feedback : التعليق النصي المكتوب وملاحظات المستخدمين.

*** الوظائف (Methods) :**

* submit : إرسال وحفظ التقييم وتعديل مؤشر ثقة الحساب المستهدف.

*** غرف المحادثة (ChatRoom) : تدير مساحات التواصل الثنائية عبر الويب سوكيت مثل:**

* id : معرّف غرفة المحادثة الرقمي.

* name : اسم أو معرّف العنوان الخاص بالغرفة.

* room_uuid : الكود العشوائي الفريد لتأمين وضمان تشفير اتصال الغرفة.

*** الوظائف (Methods) :**

* create : فتح وبدء إنشاء غرفة تواصل جديدة ومؤمنة.

* addmember : إضافة مستخدم جديد أو طرف مفاوض إلى الغرفة.

* removemember : إزالة مستخدم وإنهاء صلاحية دخوله للمحادثة.

*** الرسائل المتبادلة (Message) : تدير محتوى المراسلات النصية والملفات داخل الغرف مثل:**

* id : معرّف الرسالة الفريد في نظام قواعد البيانات.

* is_read : حقل منطقي (Boolean) لمعرفة هل تمت قراءة الرسالة أم لا.

* content :المحتوى النصي المكتوب للرسالة.

* message_type : نوع الرسالة المرسلة (نص، ملف، صورة).

*** الوظائف (Methods) :**

* sendtext : معالجة وإرسال النصوص الفورية.

* sendfile : رفع وإرسال الملفات والمستندات البرمجية.

* sendimage : معالجة وإرسال الصور الملتقطة داخل الشات.

* markasread : تحديث حالة الرسالة المقروءة للطرف الآخر تلقائياً.

*** معالجة البلوكشين (Blockchain) : الكيان البرمجي الوسيط المسؤول عن معالجة التوثيق الفردي للعقد مثل:**

* tx_hash : بصمة الهاش الرقمي الفريد الخاص بالمعاملة اللامركزية.

* block_number : رقم الكتلة الفعلي الذي تم تأكيد وحفظ العقد بداخله.

*** الوظائف (Methods) :**

* registercontract : تسجيل العقد رسمياً وكتابته على البلوكشين.

* verifycontract : التحقق ومطابقة صحة الهاش المخزن والتواقيع.

* gettransactionreceipt : جلب وإرجاع إيصال المعاملة الرسمي لتأكيد الدفع.

*** شبكة البلوكشين (BlockchainNet) : يدير إعدادات الاتصال بشبكة الإيثيريوم الموزعة (مثل Sepolia**

مثل:

* network_name : اسم الشبكة الحالية المستخدمة للتطوير والتشغيل.

* chain_id : المعرّف الرقمي الموحد الخاص بسلسلة الكتل المستهدفة.

*** الوظائف (Methods) :**

* connect : فتح خط الاتصال وتفعيل بوابة الوصول للشبكة (RPC Node) .

* sendtransaction : دفع وتمرير المعاملات الموقعة رقمياً للشبكة للمعالجة.

* waitforreceipt : الانتظار البرمجي الموقّت لحين تعدين الكتلة وإصدار الإيصال.

مخطط التتابع :

مخطط التتابع (Sequence Diagram) هو نوع من أنواع مخططات UML (Unified Modeling Language) يُستخدم لتوضيح كيفية تفاعل الكائنات أو المكونات المختلفة في نظام معين عبر الزمن. يركز مخطط التتابع على تسلسل الرسائل المتبادلة بين الكائنات، مما يساعد في فهم كيفية تنفيذ العمليات والوظائف داخل النظام.

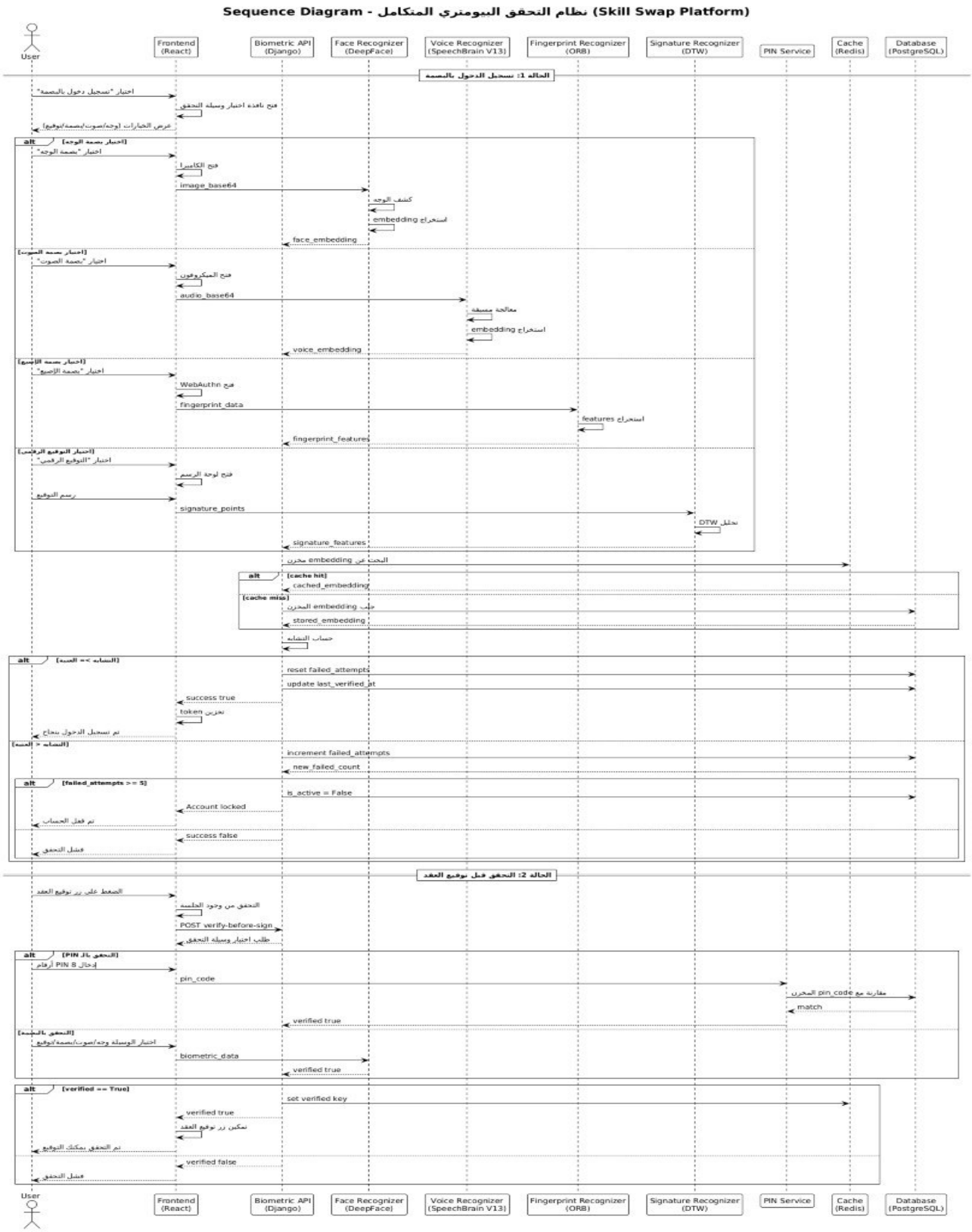
مكونات مخطط التتابع:

1. الكائنات (Objects): تمثل الكائنات أو المكونات المشاركة في التفاعل. تُعرض عادةً كأسماء مع خطوط رأسية تمثل حياتها (Lifeline).
2. الرسائل (Messages): تُظهر كيفية تواصل الكائنات مع بعضها البعض. تُرسم كخطوط أفقية بين الكائنات، مع توضيح نوع الرسالة أو الوظيفة المُستدعاة.
3. التوقيت (Timing): يُظهر الترتيب الزمني للرسائل المتبادلة. يتم ترتيب الرسائل من الأعلى إلى الأسفل، حيث تكون الرسالة الأولى في الأعلى.
4. الحياة (Lifeline): تمثل فترة حياة الكائن، وتبدأ من بداية التفاعل حتى نهايته.
5. العمليات (Activation Bars): تُظهر متى يكون الكائن نشطًا أو يقوم بعملية معينة.

استخدامات مخطط التتابع:

- تحليل النظام: لفهم كيفية تفاعل المكونات المختلفة مع بعضها.
- تصميم البرمجيات: لتوثيق كيفية تنفيذ الوظائف بشكل دقيق.
- تطوير التطبيقات: لمساعدة المطورين على فهم تدفق البيانات والرسائل بين الكائنات.

والشكل الموضح أدناه يمثل مخطط انتقال الحالة الخاص بموقعنا



الشكل (6-4) مخطط التتابع

الحالة 1 (تسجيل الدخول بالبصمة الرقمية والبيومترية):

1. يبدأ المستخدم (User) باختيار "تسجيل دخول للمنصة" عبر الواجهة الأمامية Frontend – React والتي تقوم بدورها بطلب خيارات وسيلة التحقق المتاحة من الواجهة البرمجية الخلفية (Biometric API Django) .

2. يعرض النظام للمستخدِم أربعة خيارات أساسية للمصادقة الحيوية ضمن إطار شرطي (alt) :

* عند اختيار بصمة الوجه: تُفتح الكاميرا، ويُرسل ملف الصورة المرمّز (image_base64) إلى نظام التعرف الذكي (Face Recognizer – DeepFace) لكشف الوجه واستخراج ميزات البصمة (face_embedding)

* عند اختيار بصمة الصوت: يُفتح المايكروفون، ويُرسل الملف الصوتي المرمّز (audio_base64) إلى نموذج الصوت (Voice Recognizer - SpeechBrain V13) لمعالجة الصوت واستخراج بصمته المخزنة (voice_embedding) .

* عند اختيار بصمة الإصبع: تُستدعى واجهة التوثيق (WebAuthn) لجمع بيانات الإصبع وإرسالها (fingerprint_data) لنموذج التحقق (Fingerprint Recognizer – ORB) لاستخراج الخصائص الفريدة للبصمة (fingerprint_features) .

* عند اختيار التوقيع الرقمي: تُفتح لوحة الرسم على الشاشة، ويقوم المستخدم برسم توقيعه لتنتقل نقاط الرسم (signature_points) إلى مفسر التوقيع (Signature Recognizer – DTW) لتحليل نمط التوقيع واستخراج خصائصه العميقة (signature_features)

3. بعد استخراج الـ embedding للمؤشر الحيوي، يقوم نظام Django بالبحث عنه أولاً في الذاكرة المؤقتة السريعة (Cache – Redis) :

* في حال وجوده (Cache Hit) : يتم جلب البصمة المخزنة مؤقتاً ومطابقتها وحساب التشابه فوراً.

* في حال عدم وجوده (Cache Miss) : يتجه النظام إلى قاعدة البيانات الأساسية (Database – PostgreSQL) لجلب البصمة الأصلية المخزنة (stored_embedding) وحساب المسافة والتشابه.

4. يتخذ النظام قرار التحقق بناءً على نسبة التشابه المئوية المستخرجة (alt) :

* في حال نجاح المطابقة (التشابه >= العتبة المحددة): يتم تصفير محاولات الفشل المخزنة وتحديث تاريخ آخر تحقق في قاعدة البيانات، ثم إرسال استجابة بنجاح العملية وتوليد رمز الوصول التوكِن (successtrue, token) ليتم تفعيل دخول المستخدم بنجاح.

* في حال فشل المطابقة (التشابه < العتبة): يتم زيادة عداد محاولات الفشل في قاعدة البيانات. وإذا وصلت محاولات الفشل إلى 5 محاولات أو أكثر (failed_attempts >= 5) يتم إلغاء تنشيط الحساب فوراً تلقائياً في قاعدة البيانات وعرض رسالة "تم قفل الحساب، فشل التحقق" للمستخدِم لحماية خصوصيته ومنع الاختراق.

ثانياً: الحالة 2 (التحقق الأمني قبل تفعيل توقيع العقد):

5. تبدأ هذه المرحلة عندما يضغط المستخدم على زر "توقيع العقد" في الواجهة الأمامية، حيث ترسل الواجهة طلباً للخلفية للتأكد من وجود الجلسة وصلاحيتها الحالية عبر طلب التحقق الأمني (POST verify-before-sign) .

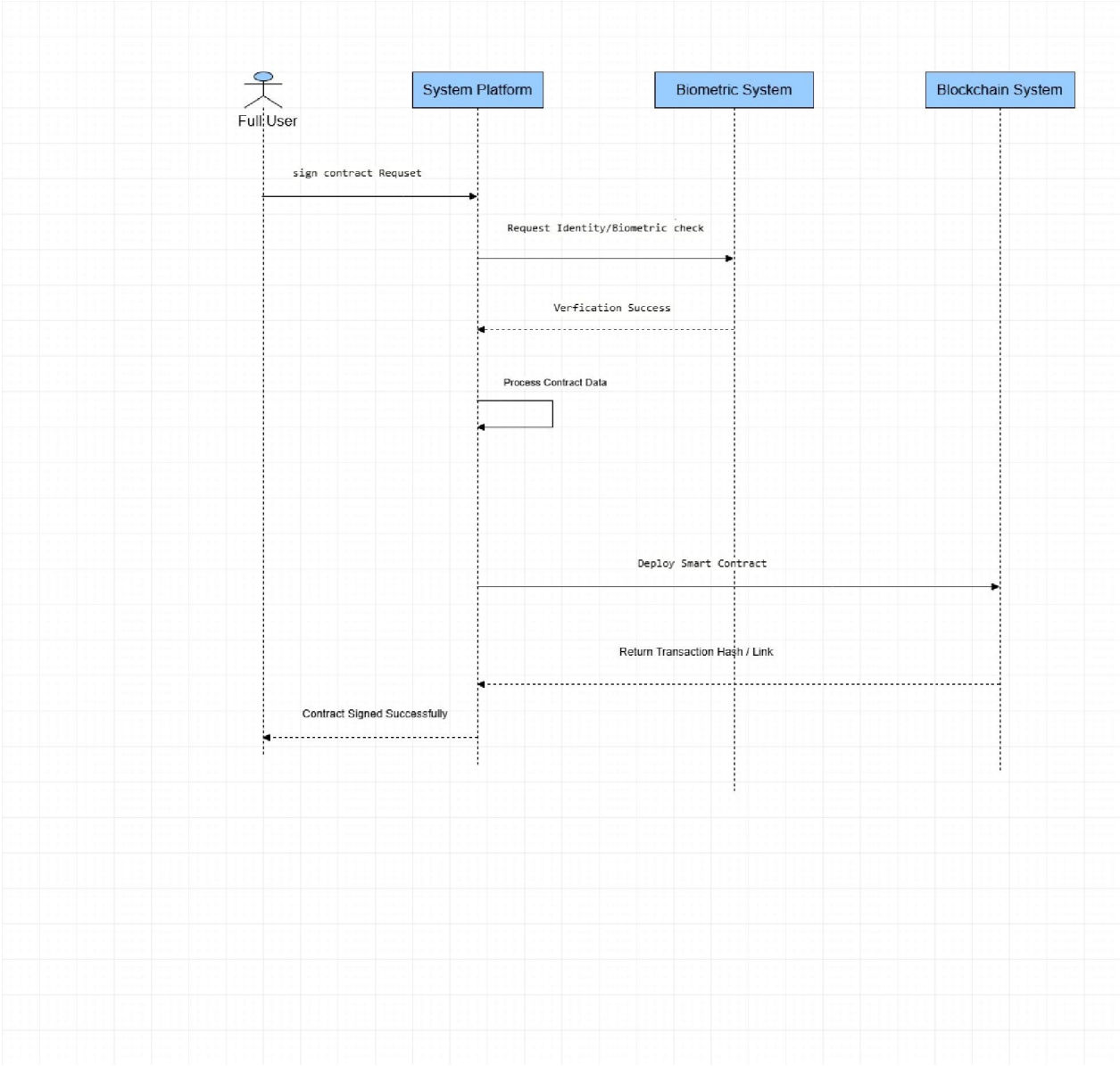
6. يطلب النظام من المستخدم اختيار طريقة التحقق المفضلة لديه لإتمام التوقيع القانوني رسميًا (alt) :

* عند اختيار التحقق برمز الـ PIN : يقوم المستخدم بإدخال رمزه السري المكون من 8 أرقام، ويُرسل الكود (pin_code) إلى خدمة الحماية (PIN Service) لمقارنته مع الرمز المشفّر والمخزن في قاعدة البيانات، وفي حال التطابق يُرجع ردّاً بالتأكيد (verified: true) .

* عند اختيار التحقق البيومتري الحسي: يختار المستخدم وسيلته الحيوية المفضلة (وجه، صوت، أو بصمة إصبع)، وتُرسل البيانات الحيوية (biometric_data) إلى خوادم معالجة البصمات، وعند تطابق الهوية ترجع استجابة مؤكدة وصحيحة للـ API (verified: true)

7. بعد نجاح وسيلة التحقق المختارة وثبوت صحتها (verified == True) يقوم النظام بحفظ مفتاح التحقق المعتمد في الـ Cache لتسريع البناء، ويُرسل رداً للواجهة يفيد بنجاح العملية والسماح الفوري للمستخدم بتوقيع وثيقة العقد وتوليد مفاتيح التشفير، أما في حال فشلها، فيتم رفض العملية وعرض رسالة فشل للمستخدم ومنعه من توقيع العقد.

بناءً على مخطط التتابع الثاني لمشروعنا والمخصص لعملية توثيق وتوقيع العقد الذكي :



الشكل (7-4) مخطط التتابع الثاني والمخصص لعملية توثيق وتوقيع العقد الذكي

. يبدأ المستخدم الكامل (Full_User) بإرسال طلب لتوقيع العقد (sign contract Request) عبر واجهة النظام الأساسية (System Platform)

2. تقوم واجهة النظام بطلب فحص الهوية والتحقق الحيوي البيومتري (Request Identity/Biometric check) من النظام الحيوي المخصص (Biometric System) لضمان أمان العملية وعدم الإنكار.

3. بعد إجراء المطابقة بنجاح، يُعيد النظام الحيوي استجابة بنجاح عملية التحقق وثبوت الهوية (Verification Success) إلى واجهة النظام.

4. عند استقبال تأكيد التحقق، تقوم واجهة النظام بمعالجة بيانات العقد داخلياً وتجهيزها برمجياً (Process Contract Data)

5. ينتقل النظام مباشرة لنشر وتفعيل العقد الذكي (Deploy Smart Contract) عبر إرساله إلى نظام كتل البلوكشين اللامركزي (Blockchain System) لتثبيته.

6. بعد معالجة المعاملة وتعدين الكتلة بنجاح، يقوم نظام البلوكشين بإرجاع بصمة المعاملة والرابط الخاص بها (Return Transaction Hash / Link) إلى واجهة النظام كإثبات تشفير دائم.

7. في النهاية، يقوم النظام بإخطار المستخدم النهائي بنجاح العملية بالكامل عبر عرض رسالة تفيد بتوقيع العقد وإتمامه آمنًا (Contract Signed Successfully)

قاعدة البيانات الخاصة بمشروعنا :

هي مجموعة من الجداول المترابطة، كل جدول مسؤول عن تخزين نوع معين من البيانات بشكل منظم ليسهل الوصول إليه، التعديل عليه، أو حذفه

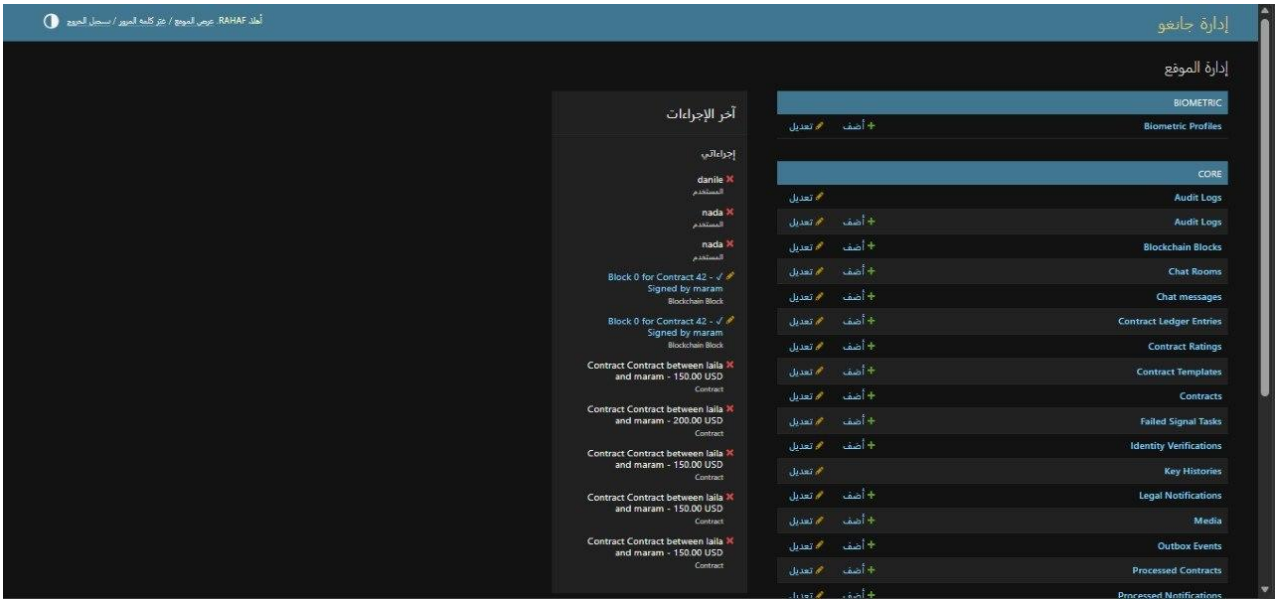
تُعد قاعدة البيانات العنصر المحوري في نظام skillswapAI ، حيث يتم تخزين كافة البيانات المتعلقة بالمستخدمين، المهارات، العقود الإلكترونية، الدردشة، البصمة البيومترية، وسجلات البلوكشين. تم تصميم قاعدة البيانات باستخدام النموذج العلائقي(Relational Model) ، حيث تم تنظيم البيانات ضمن جداول مرتبطة بعلاقات محددة لضمان سلامة البيانات وتجنب التكرار.

تم استخدام **نظام PostgreSQL** في بيئة الإنتاج، و **SQLite**في بيئة التطوير.

PostgreSQLهو نظام قواعد بيانات علائقي متقدم، تم اختياره للأسباب التالية: هو مجاني ومفتوح المصدر ولا يحتاج تراخيص مالية، يتميز بأمان عالٍ حيث يدعم تشفير البيانات والمصادقة المتقدمة، وهو قابل للتوسع ليدعم كميات كبيرة من البيانات والمستخدمين المتزامنين، كما يدعم تخزين البيانات غير المهيكلة بصيغة JSONمثل التواقيع الرقمية وبيانات البصمة، ويدعم البحث النصي المتقدم (Full-Text Search) للبحث

السريع في المنشورات والمهارات، ويتوافق مع معايير ACID التي تضمن سلامة البيانات وموثوقيتها حتى في حالات الأعطال

أما **SQLite**فتم استخدامه في بيئة التطوير فقط، وهو نظام قواعد بيانات خفيف يعمل بدون خادم (Serverless)، مما يعني أنه يعمل داخل التطبيق دون الحاجة لخادم مستقل. لا يحتاج إلى تثبيت أو إعداد معقد، فقط ملف قاعدة بيانات واحد يتم الوصول إليه مباشرة. هو متوافق مع جميع أنظمة التشغيل–Cross) (Platform، وخفيف الوزن (Small Footprint) مما يجعله مناسباً للتطوير السريع والاختبار، كما يدعم المعاملات ACID ويوفر دعم واسع للغة.SQL



الشكل (8-4) قاعدة البيانات

محتويات قاعدة البيانات :

- **جدول المستخدمين(Users) :**

الوظيفة: يُستخدم لتخزين وإدارة بيانات الحسابات والمعلومات الشخصية للمستخدمين، بالإضافة إلى بيانات المصادقة والتوقيعات البيومترية.

المحتويات:

id (Integer, Primary Key, Auto-increment, Not Null): معرف فريد لكل مستخدم في النظام.
username (Varchar, Not Null, Unique): اسم المستخدم الفريد لتسجيل الدخول والتحديد داخل المنصة.

- email (Varchar, Not Null, Unique) : البريد الإلكتروني الخاص بالمستخدم للتواصل والإشعارات.
- password_hash (Varchar, Not Null) : كلمة المرور الخاصة بالمستخدم مخزنة بشكل مشفر (Hashed) لضمان الأمان.
- first_name & last_name (Varchar, Nullable): الاسم الأول والأخير للمستخدم.

• biometric_hash (Text/Varchar, Nullable): البصمة الرقمية أو الهاش المشفر الخاص بالبيانات البيومترية لاستخدامه في المصادقة والتوقيع.
• role (Varchar, Not Null): دور المستخدم داخل المنصة (مثل: مستقل/مقدم خدمة، عميل/طالب خدمة، أو مدير نظام).
• created_at (DateTime, Not Null): تاريخ ووقت إنشاء الحساب.
التفسير: يحتوي الجدول على بيانات الاعتماد الحيوية للتحقق من هوية الحساب للوصول إلى المنصة.

- **جدول (User Profiles) :**

الوظيفة: يُستخدم لتخزين الملف الشخصي، الإحصائيات، ودرجة الثقة للمستخدم.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف فريد للملف الشخصي.
user_id (Integer, Foreign Key, Not Null) : معرف المستخدم المرتبط بجدول auth_user
bio (Text, Nullable) : السيرة الذاتية للمستخدم.
trust_score (Float, Default) : درجة الثقة المحسوبة للمستخدم.
completion_rate (Float, Default) : نسبة إنجاز المهام للعقود.
rating_avg (Float, Default) : متوسط التقييم العام للمستخدم.
pin_hash (Varchar, Nullable) : رمز الـ PIN المكون من 8 أرقام والمشفر أمنياً.
التفسير: يربط البيانات التكميلية والتنفيذية للمستخدم مع تقييم أداء الحساب ودرجة الثقة (Trust Score)

- **جدول (Skills) :**

الوظيفة :يُستخدم لبناء القاموس الدليلي للمهارات المتاحة للتداول في المنصة.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف فريد للمهارة.
name (Varchar, Not Null) : اسم المهارة.
category (Varchar, Not Null) : التصنيف أو الفئة التي تتبع لها المهارة.
demand_level (Integer, Not Null) : مستوى الطلب على المهارة (من 0 إلى 100).
avg_price (Decimal, Not Null): السعر الاسترشادي المتوسط للمهارة.
التفسير :يُخزن المهارات البرمجية والتقنية المعترف بها داخل النظام لتسهيل عمليات البحث والمطابقة.

- **جدول (Skill Posts) :**

الوظيفة: يُستخدم لتخزين منشورات العرض والطلب للمهارات التي يقدمها المستخدمون.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف المنشور.
creator_id (Integer, Foreign Key, Not Null) : معرف منشئ العرض (مرتبط بـ auth_user).
skill_id (Integer, Foreign Key, Not Null) : المهارة المعروضة (مرتبط بـ Skills).
title (Varchar, Not Null) : عنوان منشور المهارة.
description (Text, Not Null) : تفاصيل العرض والشروط.
price (Decimal, Not Null) : السعر المطروح للمهارة.
status (Varchar, Not Null) : حالة المنشور (مفتوح open / مغلق closed)
التفسير: يربط العروض والمقابل المالي بين مقدم الخدمة والمهارة المطلوبة في السوق.

- **جدول (Contracts) :**

الوظيفة: يُستخدم للحفظ والتحكم في دورة حياة العقود الإلكترونية بين الأطراف.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف فريد للعقد.
client_id (Integer, Foreign Key, Not Null) : معرف العميل صاحب الطلب.

freelancer_id (Integer, Foreign Key, Not Null) : معرف المستقل المنفذ.
total_amount (Decimal, Not Null) : المبلغ المالي الإجمالي للعقد.
status (Varchar, Not Null) : حالة العقد (معلق pending / نشط active / مكتمل completed)
client_signature (Text, Nullable) : توقيع العميل الرقمي المشفر بـ RSA.
freelancer_signature (Text, Nullable) : توقيع المستقل المشفر بـ RSA.
contract_hash (Varchar, Not Null) : تجزئة العقد باستخدام خوارزمية SHA-256.
blockchain_tx_hash (Varchar, Nullable): عنوان المعاملة على شبكة البلوكشين.
التفسير: يعتبر محور العمليات الإلكترونية للربط بين الأطراف وتأمين الاتفاقيات برمجياً وتوثيقها بـ RSA والبلوكشين.

- **جدول (Contract Ratings) :**

الوظيفة: يُستخدم لتخزين تقييمات وتعليقات الأطراف بعد إتمام العقد.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف التقييم.
contract_id (Integer, Foreign Key, Not Null) : العقد المقيّم.
rated_by_id (Integer, Foreign Key, Not Null) : معرف المستخدِم المُقَيِّم.
rated_user_id (Integer, Foreign Key, Not Null) : معرف المستخدِم المُقَيَّم.
rating (Float, Not Null) : الدرجة الممنوحة (من 1 إلى 5).
feedback (Text, Nullable) : الملاحظات والتعليقات المكتوبة.
التفسير: يُعتمد عليه في حساب درجات الثقة (Trust Score) وتعديل خوارزميات الاقتراح بناءً على أداء الأطراف.

- **جدول (Chat Rooms) :**

الوظيفة: يُستخدم لإنشاء وإدارة غرف المحادثة للتفاوض بين العميل والمستقل.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف الغرفة.
room_uuid (UUID, Not Null, Unique) : معرف عالمي فريد للغرفة.
room_type (Varchar, Not Null) : نوع المحادثة والتفاوض.
created_at (DateTime, Not Null) : تاريخ وساعة إنشاء الغرفة.
التفسير: يمثل المساحة المخصصة للتواصل المباشر والربط بين طرفين عبر بروتوكول WebSockets.

- **جدول (Chat Messages) :**

الوظيفة: يُستخدم لتخزين الرسائل المتبادلة في المحادثات بشكل مشفر أمنياً.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف الرسالة.
room_id (Integer, Foreign Key, Not Null) : معرف الغرفة التابع لها الرسالة.
sender_id (Integer, Foreign Key, Not Null) : معرف مرسل الرسالة.
encrypted_content (Text, Not Null) : النص المشفر بخوارزمية Fernet (AES-128).
message_type (Varchar, Not Null) : نوع الرسالة (نص، صورة، فيديو، مستند).
status (Varchar, Not Null) : حالة الرسالة (مرسلة sent / مستلمة delivered / مقروءة read)
التفسير: يُؤمن حماية الخصوصية عن طريق تخزين نصوص ورسائل المحادثة في الحالة المشفرة داخل القاعدة.

- **جدول (Blockchain Blocks) :**

الوظيفة: يُستخدم لربط العقود بمعاملات التوثيق على شبكة البلوكشين (Ethereum Sepolia)
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف السجل.
contract_id (Integer, Foreign Key, Not Null) : معرف العقد الموثق.
transaction_hash (Varchar, Not Null) : الهاش الخاص بالمعاملة على réseau Ethereum.
is_confirmed (Boolean, Default: False) : تأكيد وصول المعاملة واستقرارها بالبلوكشين.
network (Varchar, Not Null) : اسم شبكة الاختيار (sepolia).
التفسير: يوفر مرجعاً صلباً لإثبات صحة العقد وربطه بسجلات اللامركزية العامة غير القابلة للتعديل.

- **جدول (Users Keys) :**

الوظيفة: يُستخدم لإدارة وتخزين مفاتيح التوقيع الرقمي RSA وعناوين المحافظ.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف سجل المفتاح.
user_id (Integer, Foreign Key, Not Null) : معرف المستخدم صاحب المفاتيح.
private_key (EncryptedTextField, Not Null) : المفتاح الخاص المشفر.
public_key (Text, Not Null) : المفتاح العام المتاح للتحقق.
eth_wallet_address (Varchar, Not Null) : عنوان محفظة الإيثيريوم المربوطة.
التفسير: يُعد الجزء الأمني الحساس المسند للتحقق من هوية المفاتيح المسؤولة عن توقيع العقود رقمياً.

- **جدول (Key Histories) :**

الوظيفة: يُستخدم لأرشفة المفاتيح القديمة عند تدوير المفاتيح الأمنية للحساب.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف الأرشيف.
user_key_id (Integer, Foreign Key, Not Null) : معرف سجل المفتاح الأصلي.
private_key (EncryptedTextField, Not Null) : المفتاح القديم المؤرشف.
rotated_at (DateTime, Not Null) : تاريخ ووقت تغيير وتدوير المفتاح.
التفسير: يضمن عدم ضياع التوقيعات القديمة على العقود وتتبع سلاسل المفاتيح السابقة للحسابات.

- **جدول (Identity Verifications) :**

الوظيفة: يُستخدم لإدارة طلبات توثيق الهوية بالوثائق الرسمية والصور.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف طلب التحقق.
user_id (Integer, Foreign Key, Not Null) : معرف المستخدم صاحب الطلب.
verification_method (Varchar, Not Null) : نوع الوثيقة (بطاقة شخصية، جواز سفر، رخصة).
document_front (ImageField, Not Null) : صورة الوجه الأمامي للوثيقة.
verification_status (Varchar, Not Null) : حالة التوثيق (قيد الانتظار pending / مقبول approved / مرفوض rejected)
التفسير: يحفظ ملفات إثبات الشخصية لضمان سلامة التعاملات والحد من الحسابات وهمية الهوية.

- **جدول (Verification Audit Logs) :**

الوظيفة: يُستخدم كسجل تتبع وتدقيق لجميع الإجراءات المتخذة على طلبات الهوية.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف السجل.
verification_id (Integer, Foreign Key, Not Null) : الطلب المعالج.
action (Varchar, Not Null) : نوع الإجراء (موافقة، رفض، طلب تعديل).

performed_by_id (Integer, Foreign Key, Not Null) : معرف المشرف المنفذ للإجراء.
timestamp (DateTime, Not Null) : التوقيت الزمني الدقيق للعملية.
التفسير: يُتيح للإدارة مراقبة عملية توثيق الهويات والرجوع للتعديلات التاريخية للحسابات.

- **جدول (Legal Notifications) :**

الوظيفة: يُستخدم لإدارة وتخزين الإشعارات والتنبيهات ذات الطابع القانوني.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف الإشعار.
contract_id (Integer, Foreign Key, Not Null) : معرف العقد ذي الصلة.
user_id (Integer, Foreign Key, Not Null) : المستلم للإشعار.
notification_type (Varchar, Not Null) : تصنيف الإشعار القانوني.
message (Text, Not Null) : نص التنبيه أو الإخطار.
read_at (DateTime, Nullable) : تاريخ وقراءة المستلم للإشعار.
التفسير: يوثق الإخطارات الرسمية المتبادلة بين النظام والأطراف للتأكد من الاطلاع المتبادل.

- **جدول (Media) :**

الوظيفة: يُستخدم لحفظ وإدارة الوسائط والملفات المرفوعة وربطها بـ IPFS.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف الملف.
file (FileField, Not Null) : مسار وتفاصيل الملف المرفوع.
uploaded_by_id (Integer, Foreign Key, Not Null) : المستخدم الممارس للرفع.
file_type (Varchar, Not Null) : صيغة امتداد الملف.
ipfs_cid (Varchar, Nullable) : المعرف الفريد للمحتوى على شبكة IPFS اللامركزية.
التفسير: يعمل كنقطة ربط بين التخزين المحلي والنظام التخزيني اللامركزي لتأمين الملفات المرفقة.

- **جدول (Contract Templates) :**

الوظيفة: يُستخدم لتخزين قوالب عقود جاهزة ومعتمدة لتسهيل الإنشاء.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف القالب.
name (Varchar, Not Null) : اسم نموذج العقد.
template_content (Text, Not Null) : النصوص والبنود القانونية الثابتة.
is_approved (Boolean, Default: True) : اعتماد القالب للاستخدام العام.
التفسير: يوفر نماذج اتفاقيات مسبقة الإعداد لتسريع إجراءات التوافق وصياغة العقد.

- **جدول (Contract Ledger Entries) :**

الوظيفة: يُستخدم كسجل أحداث العقد المتسلسل (دفتر أستاذ) كبديل متسلسل داخل القاعدة.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف القيد.
entry_id (Varchar, Not Null, Unique) : معرف القيد الفريد.
contract_id (Integer, Foreign Key, Not Null) : معرف العقد المرتبط.
entry_type (Varchar, Not Null) : نوع الحدث المسجل.
previous_hash (Varchar, Not Null) : قيمة الهاش للقيد السلسلي السابق.
current_hash (Varchar, Not Null) : قيمة الهاش للقيد الحالي.
التفسير: يطبق مفهوم السلسلة المشفرة (Blockchain-like) داخل قاعدة البيانات لضمان عدم التلاعب بالسجلات.

- **جدول (Outbox Events) :**

الوظيفة: يُستخدم لتخزين أحداث النظام غير المتزامنة لضمان المعالجة عبر .Celery
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف الحدث.
event_type (Varchar, Not Null) : نوع الحدث المثار.
payload (JSON, Not Null) : الحمولة والبيانات المرفقة بالحدث.
status (Varchar, Not Null) : حالة التنفيذ (قيد الانتظار pending / مكتمل completed / فشل failed)
التفسير: يطبق نمط Outbox Pattern لضمان موثوقية تنفيذ المهام الخلفية المعقدة بآمان.

- **جدول (Failed Signal Tabs / DLQ) :**

الوظيفة: يُستخدم كقائمة انتظار للمهام والأحداث التي فشلت معالجتها (.Dead Letter Queue)
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف السجل.
task_name (Varchar, Not Null) : اسم المهمة المتوقفة.
error (Text, Not Null) : تفاصيل الخطأ البرمجي المسبب للفشل.
resolved (Boolean, Default: False) : بيان حالة مراجعة وحل المشكلة.
التفسير: يحتفظ بالعمليات غير الناجحة لمساعدة المطورين على تصحيح الأخطاء وإعادة تشغيل العمليات.

- **جدول (Processed Contracts) :**

الوظيفة: يُستخدم لمنع التكرار في معالجة العقود (.Idempotency)
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) :معرف السجل.
contract_id (Integer, Unique, Not Null) : معرف العقد المعالج.
processed_at (DateTime, Not Null) : وقت تاريخ تنفيذ المعالجة.
التفسير: يضمن عدم تكرار تنشيط أو توثيق العقد أكثر من مرة في حال أُعيد إرسال الطلب أوتوماتيكياً.

- **جدول (Processed Notifications) :**

الوظيفة: يُستخدم لمنع التكرار في إرسال الإشعارات للمستخدم.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : المعرف.
notification_key (Varchar, Not Null, Unique) :المفتاح المرجعي الفريد للإشعار.
created_at (DateTime, Not Null) : وقت وتاريخ المعالجة والإرسال.
التفسير: يمنع إزعاج المستخدم بتكرار وصول ذات التنبيهات أكثر من مرة عند التحديث.

- **جدول (Skill update logs) :**

الوظيفة: يُستخدم لتتبع وسجل تحديثات المهارات للتحكم بمعدل الطلبات (.Rate Limiting)
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : المعرف.
skill_id (Integer, Foreign Key, Not Null) : المهارة المستهدفة بالعد والتعديل.
hour_key (Varchar, Not Null) : المفتاح المرجعي للتحديد الزمني.
created_at (DateTime, Not Null) : وقت تسجيل التحديث.
التفسير: يحمي النظام من الهجمات أو التعديلات المفرطة على بيانات المهارات في زمن قصير.

- **جدول (Biometric Audit Logs) :**

الوظيفة: يُستخدم كسجل أمني لتدقيق وتتبع كافة محاولات البصمة البيومترية.
المحتويات:
id (Integer, Primary Key, Auto-increment, Not Null) : معرف محاولة المصادقة.
user_id (Integer, Foreign Key, Not Null) : معرف المستخدم.
action (Varchar, Not Null) : نوع الإجراء (تحقق verify / تسجيل enroll)
status (Varchar, Not Null) : نتيجة المحاولة (نجاح success / فشل failed)
confidence_score (Float, Not Null) :درجة المطابقة والدقة الناتجة عن نموذج الذكاء الاصطناعي.
التفسير: يوفر بيئة أمنية شفافة للتحقق من كفاءة ومصداقية عمليات التحقق الحيوية للوجه والصوت.

العلاقات بين الجداول:

users ↔user_profiles :علاقة واحد إلى واحد؛ يمكن لمستخدم واحد أن يمتلك ملفاً شخصياً واحداً فقط.
users ↔user_keys : علاقة واحد إلى واحد؛ يمكن لمستخدم واحد أن يمتلك مفاتيح تشفير ومحفظة واحدة.
users ↔ skill_posts : علاقة واحد إلى متعدد؛ يمكن لمستخدم واحد أن ينشر عدة منشورات مهارات.
skills ↔skill_posts : علاقة واحد إلى متعدد؛ يمكن لمهارة واحدة أن ترتبط بعدة منشورات مهارات.
users ↔contracts : علاقة واحد إلى متعدد؛ يمكن لمستخدم واحد أن يكون طرفاً (كعميل أو كمستقل) في عدة عقود.
contracts ↔contract_ratings : علاقة واحد إلى واحد؛ يمكن لعقد مكتمل واحد أن يحصل على تقييم واحد فقط.
contracts ↔blockchain_blocks : علاقة واحد إلى واحد؛ يمكن لعقد موثق واحد أن يرتبط بمعاملة توثيق واحدة على البلوكشين.
contracts ↔chat_rooms : علاقة واحد إلى واحد؛ يمكن لعقد واحد أن ينشأ عن طريق غرفة دردشة واحدة.
chat_rooms ↔ chat_messages: علاقة واحد إلى متعدد؛ يمكن لغرفة دردشة واحدة أن تحتوي على عدة رسائل.
users ↔chat_messages : علاقة واحد إلى متعدد؛ يمكن لمستخدم واحد أن يرسل عدة رسائل.
contracts↔contract_audit_logs : علاقة واحد إلى متعدد؛ يمكن لعقد واحد أن يمتلك عدة سجلات تدقيق وتغيير حالات.

الفصل الخامس :

واجهات الموقع

Chapter Five :

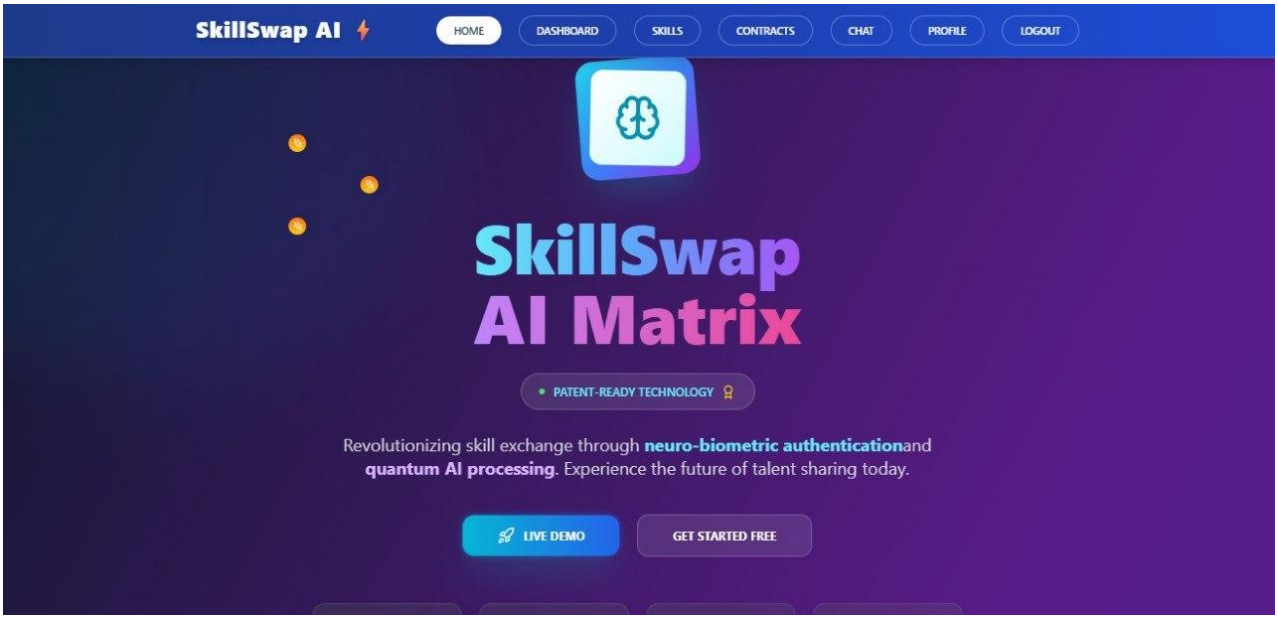
Project Interfaces

مقدمة :

يهدف هذا الفصل إلى توثيق واجهات موقعنا (BioSign X) الذي تم تصميمه لتوفير تجربة استخدام آمنة وسلسة لمستخدمي منصة تبادل المهارات. سنستعرض في هذا الفصل تصميم الواجهات، الميزات الرئيسية، وأهم العناصر التفاعلية التي تسهم في تعزيز تجربة المستخدم، كما سنُسلط الضوء على كيفية تيسير الوصول إلى الخدمات والمهارات المتاحة، وإدارة العقود والدردشة اللحظية، وتوثيق البيانات بشكل موثوق.

نسعى إلى ضمان أن يكون موقعنا ليس فقط جذاباً بصرياً، ولكن أيضاً سهل الاستخدام، مما يتيح للمستخدمين والمستقلين إتمام اتفاقاتهم وتوثيق عقودهم دون أي عوائق. سيتم تناول كل واجهة بشكل مفصل، بما في ذلك الأزرار، القوائم، صفحات المهارات والعقود، والعديد من العناصر الأخرى التي تشكل هيكل الموقع.

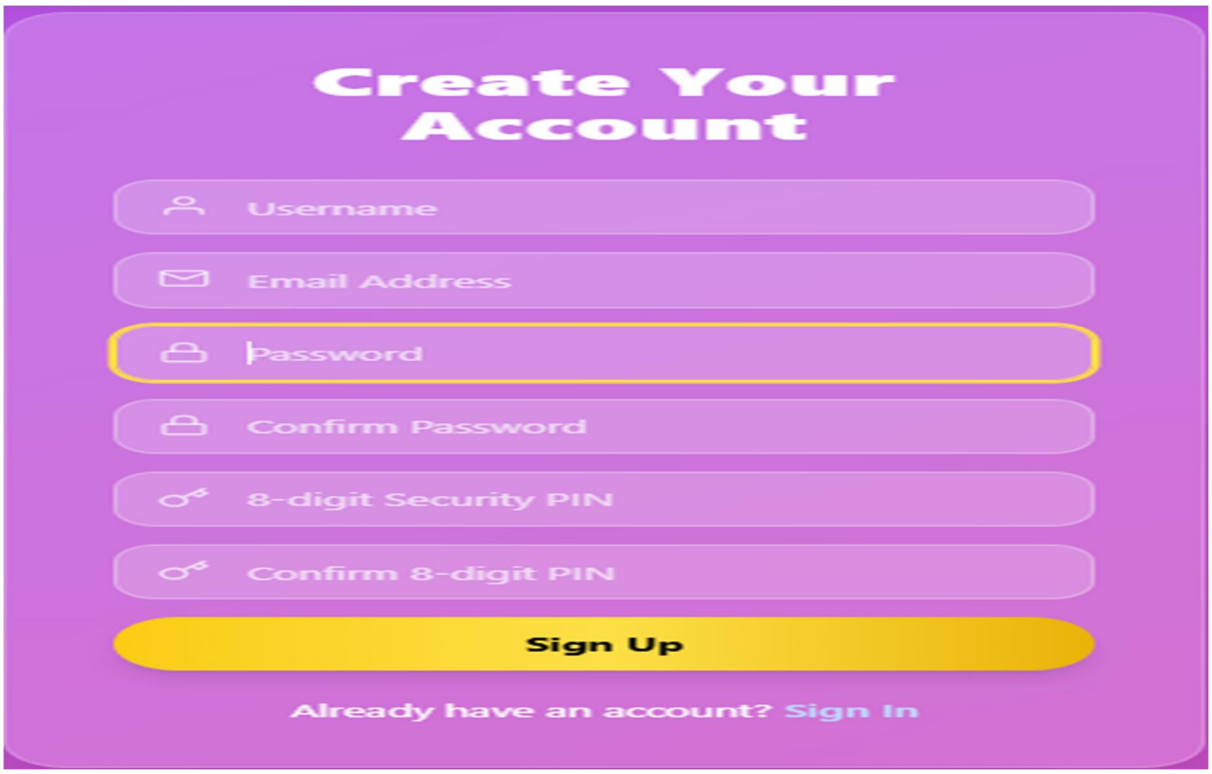
الواجهة الرئيسية :



الشكل(9-5) الواجهة الرئيسية

ُمثل هذه الواجهة نقطة الوصول الأولى للمستخدم عند دخوله إلى المنصة، حيث تُقدّم تعريفا شاملاً بهوية منصة SkillSwap AI وتقنياتها المتقدمة. تم تصميمها بأسلوب عصري جذاب يعكس التوجه الذكي والأمني للنظام، مع توفير شريط navigation خارجي يتيح الوصول السريع لجميع أجزاء المنصة.

واجهة التسجيل(Register) :



الشكل (10-5) واجهة التسجيل

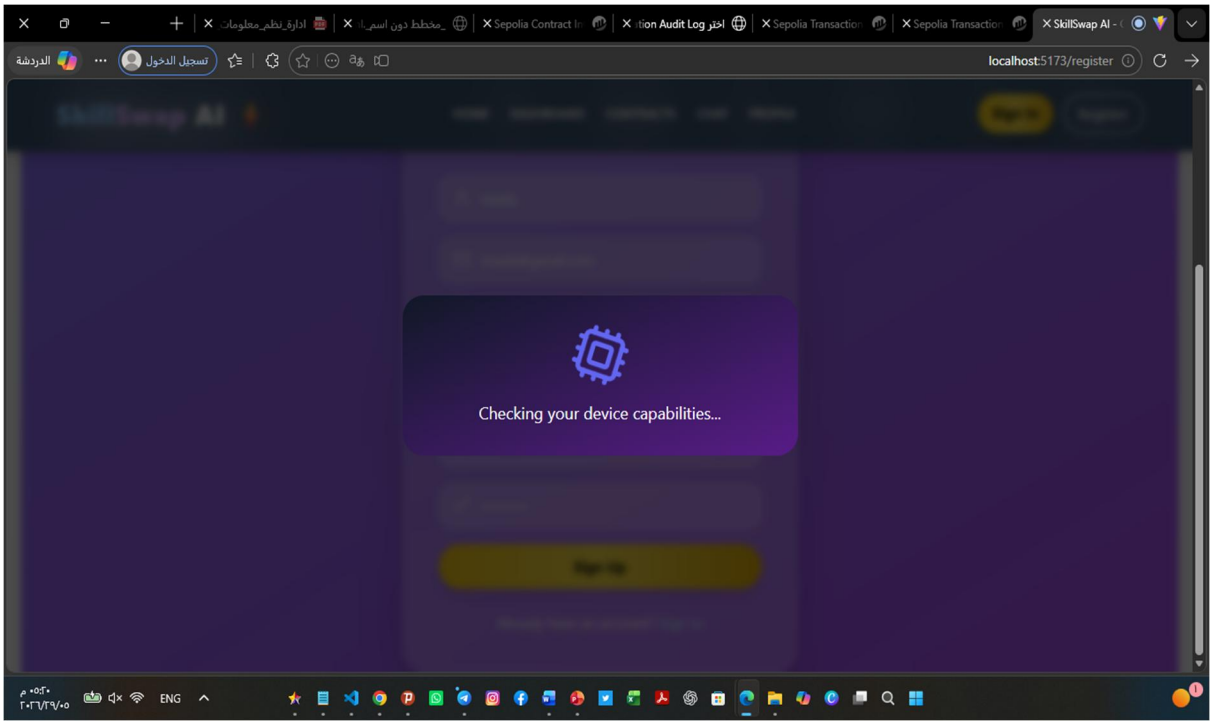
الخاصية	الوصف
الغرض	إنشاء حساب جديد للمستخدمين الجدد
المستخدمون	الزوار (غير المسجلين)
المبادئ المحققة	التحقق من التسلسل، التحقق من النطاق، إرشادات واضحة

قواعد التحقق:

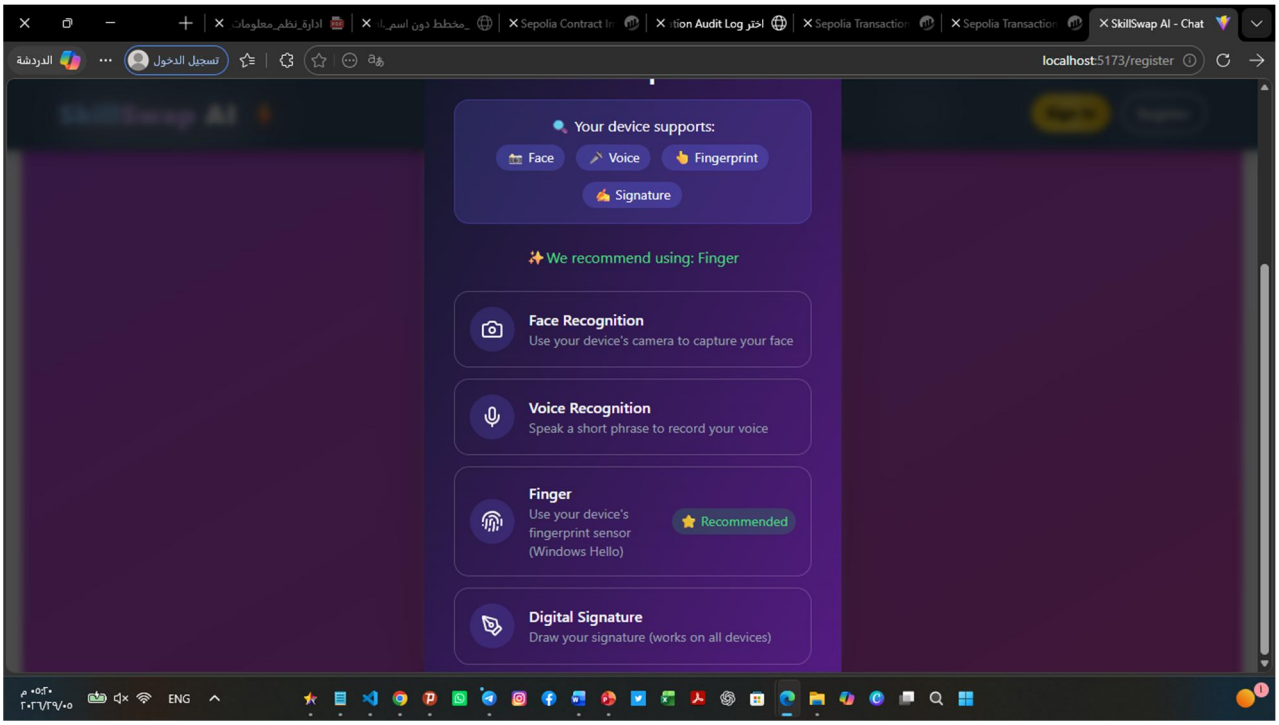
- **تطابق كلمة المرور مع تأكيدها**

- PINمكون من 8 أرقام بالضبط
- تطابق PIN مع تأكيده

واجهة كشف إمكانيات الجهاز :



الشكل(11-5) واجهة كشف إمكانيات الجهاز



الشكل (12-5) واجهة إمكانيات الجهاز

الخاصية	الوصف
الغرض	اكتشاف إمكانيات جهاز المستخدم (كاميرا، ميكروفون، بصمة)
المستخدمون	المستخدمون الجدد بعد التسجيل
المبادئ المحققة	التفكير بطريقة المستخدم، تقديم خيارات مناسبة للجهاز

محتويات الواجهة:

- إشعار "جاري كشف إمكانيات جهازك"...
- عرض الوسائل المتاحة (وجه، صوت، بصمة، توقيع)
- توصية بالوسيلة الأنسب للجهاز
- زر لتحديد وسيلة التسجيل

ماذا تفعل؟

- تفحص وجود كاميرا ← تعرض خيار "بصمة الوجه"
- تفحص وجود ميكروفون ← تعرض خيار "بصمة الصوت"
- تفحص وجود مستشعر بصمة ← تعرض خيار "بصمة الإصبع"

- **تعرض خيار "التوقيع الرقمي" دائماً (بديل آمن)**

واجهة تسجيل الوجه (Face Recognition Enrollment) :

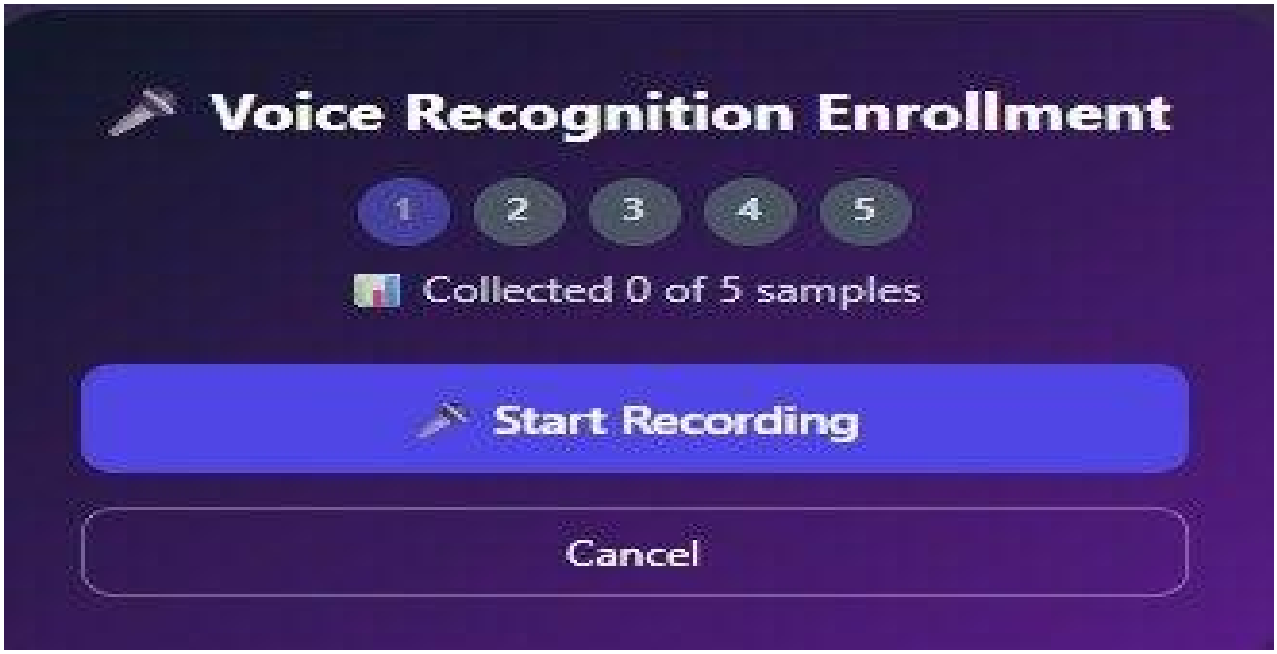


الشكل (13-5) واجهة تسجيل الوجه

وظيفتها:

التقاط صورة الوجه: التقاط صورة مباشرة للمستخدم عبر الكاميرا عند الضغط على زر .Capture Photo
معالجة واستخراج خصائص الوجه: تُستخدم الصورة لالتقاط ملامح الوجه وتوليد المتجهات الخاصة بها (Face Embeddings) باستخدام نموذج التمييز (مثل DeepFace) ، ليتم حفظها مشفرة في قاعدة البيانات لاستخدامها مستقبلاً في عمليات المطابقة والمصادقة البيومترية

واجهة تسجيل الصوت :



الشكل (14-5) واجهة تسجيل الوجه

وظيفتها:

جمع العينات الصوتية: تسجيل 5 عينات صوتية للمستخدم (Collected 0 of 5 samples) عند الضغط على زر Start Recording

باستخدام مكتبات مثل بناء البصمة الصوتية: استخلاص الخصائص الفريدة لصوت المستخدم

تُخزن في النظام (Voice Print/Embeddings) وتوليد بصمة صوتية رقمية (SpeechBrain

لاستخدامها في التحقق من هوية المستخدم لاحقاً

واجهة التوقيع اليدوي :



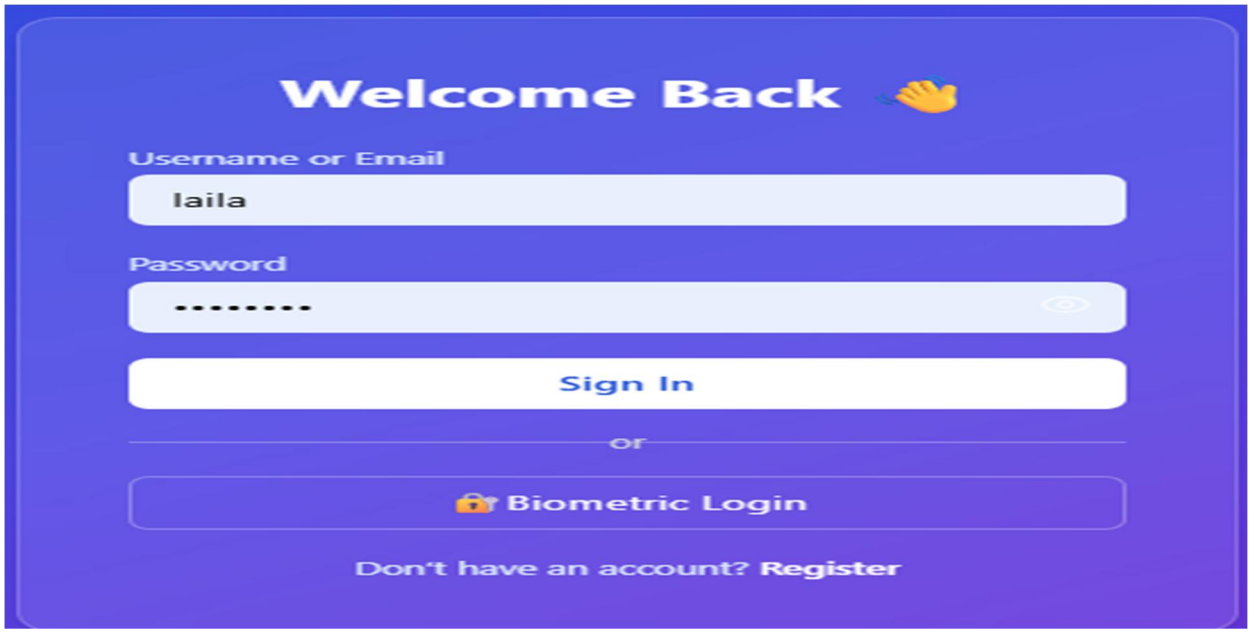
الشكل (15-5) واجهة التوقيع اليدوي

الوظيفة الأساسية للواجهة

إدخال التوقيع اليدوي: تتيح للمستخدم رسم توقيعه اليدوي مباشرةً على الشاشة (باستخدام الفأرة أو اللمس).

معالجة وحفظ التوقيع: يتم التقاط أحداث الرسم (سلسلة النقاط والمسارات) وتحويلها إلى صورة/بيانات رقمية تُستخدم كأحد عوامل المصادقة البيومترية لتأكيد الهوية أو لتوقيع العقود داخل المنصة.

واجهة تسجيل الدخول :



الشكل (16-5) واجهة تسجيل الدخول

الخاصية	الوصف
الغرض	السماح للمستخدمين المسجلين بالدخول إلى المنصة
المستخدمون	جميع المستخدمين (المسجلون مسبقاً)
المبادئ المحققة	وضوح التسميات، عدد خيارات محدود، دعم المبتدئين والمحترفين

محتويات الواجهة:

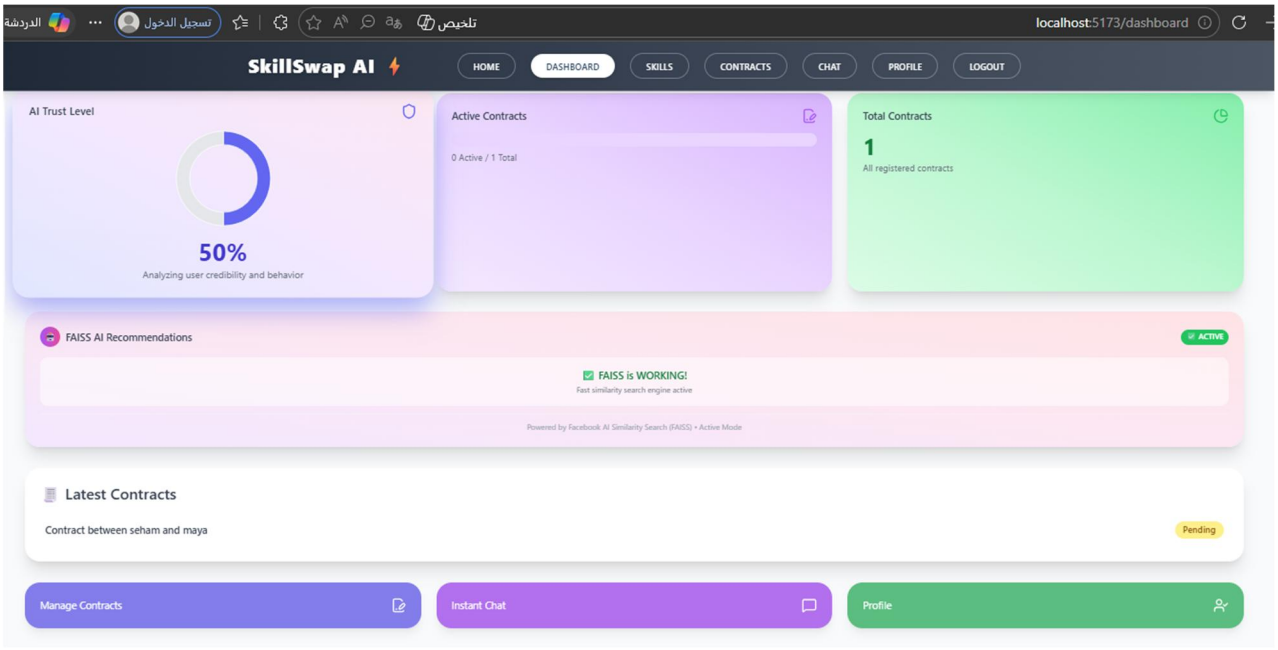
- حقل إدخال البريد الإلكتروني أو اسم المستخدم
- حقل إدخال كلمة المرور (مع إمكانية إظهار/إخفاء)
- زر "تسجيل الدخول" (للدخول بالبريد وكلمة المرور)
- زر "تسجيل دخول بالبصمة" (للمستخدمين المحترفين)
- رابط "إنشاء حساب جديد" (للتوجه إلى صفحة التسجيل)

ماذا تظهر؟

- رسالة خطأ إذا كانت البيانات غير صحيحة
- مؤشر تحميل أثناء معالجة الطلب

عندما يتم التحقق بنجاح ينتقل المستخدم إلى لوحة التحكم

واجهة لوحة التحكم :



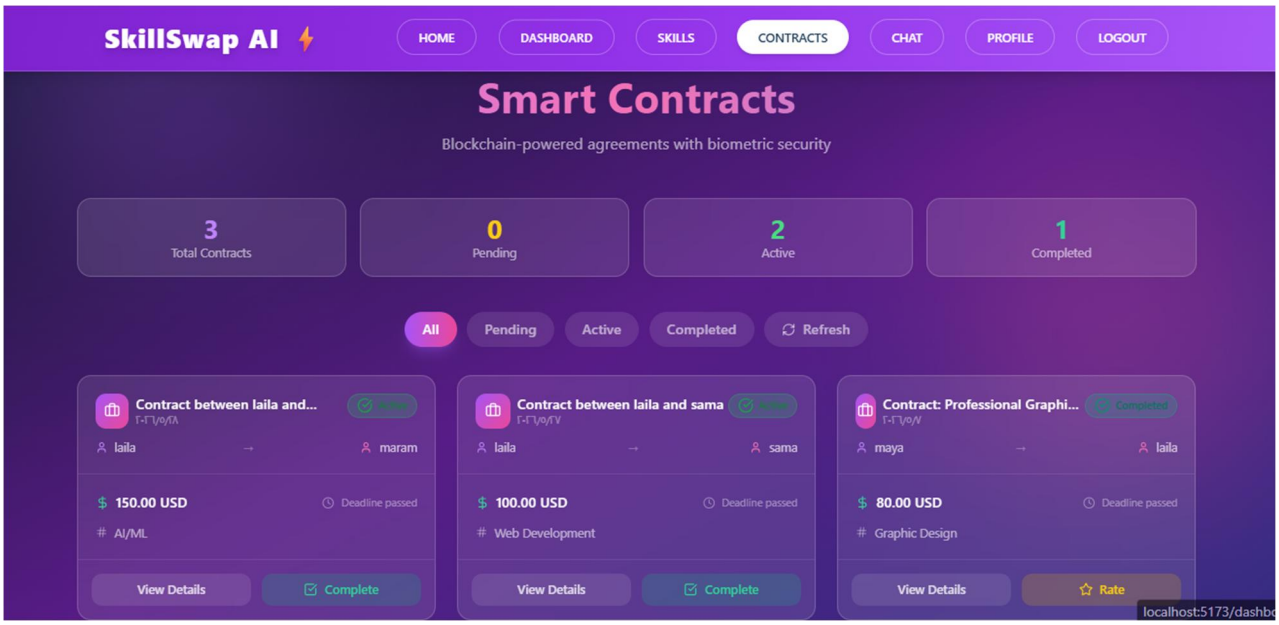
الشكل (17-5) واجهة لوحة التحكم

الخاصية	الوصف
الغرض	عرض إحصاءات المستخدم وعقوده بشكل مرئي
المستخدمون	جميع المستخدمين المسجلين
المبادئ المحققة	عرض إحصاءات بيانية، اختصارات سريعة

محتويات الواجهة:

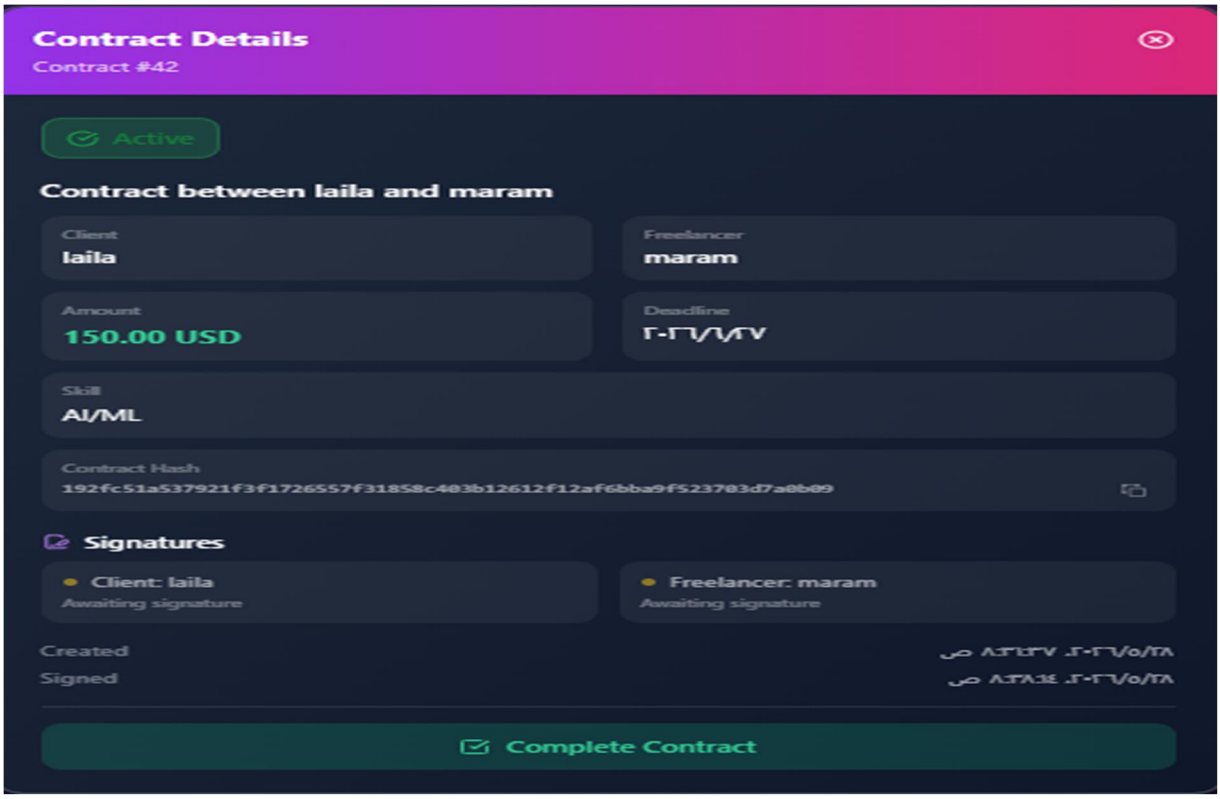
- بطاقة "مستوى الثقة (Trust Score) "بشكل دائري(PieChart)
- بطاقة "العقود النشطة" (عدد العقود النشطة)
- بطاقة "إجمالي العقود" (مجموع العقود)
- قسم "آخر العقود" (قائمة بأحدث 5 عقود)
- أزرار اختصار:
  - "إدارة العقود" ← ينتقل إلى صفحة العقود
  - "الدردشة" ← ينتقل إلى صفحة الدردشة
  - "الملف الشخصي" ← ينتقل إلى صفحة الملف الشخصي

- نلاحظ وجود ثلاث أزرار
- الزر الأول mange contractsتاخذنا االى واجهة العقود حيث تظهر في هذه الواجهة العقود وحالة هذاول تفاصيل العقد واله العقود فاذا قام طرف واحد بتوقيع العقد فيظهر بحالة pendingأما في حال كلا الطرفين وقع العقد فيصبح active وكل عقد يتضمن الأطراف والمدة والتاريخ والمبلغ التفق عليه وكل عقد يملك زرين الأول viewDetalis ياخذنا لتفاصيل العقد وزر completeوهو اكمال العقد وعند اكمال العقد يظهر زر Rateلتقييم العقد سنظهر ذلك بالواجهات



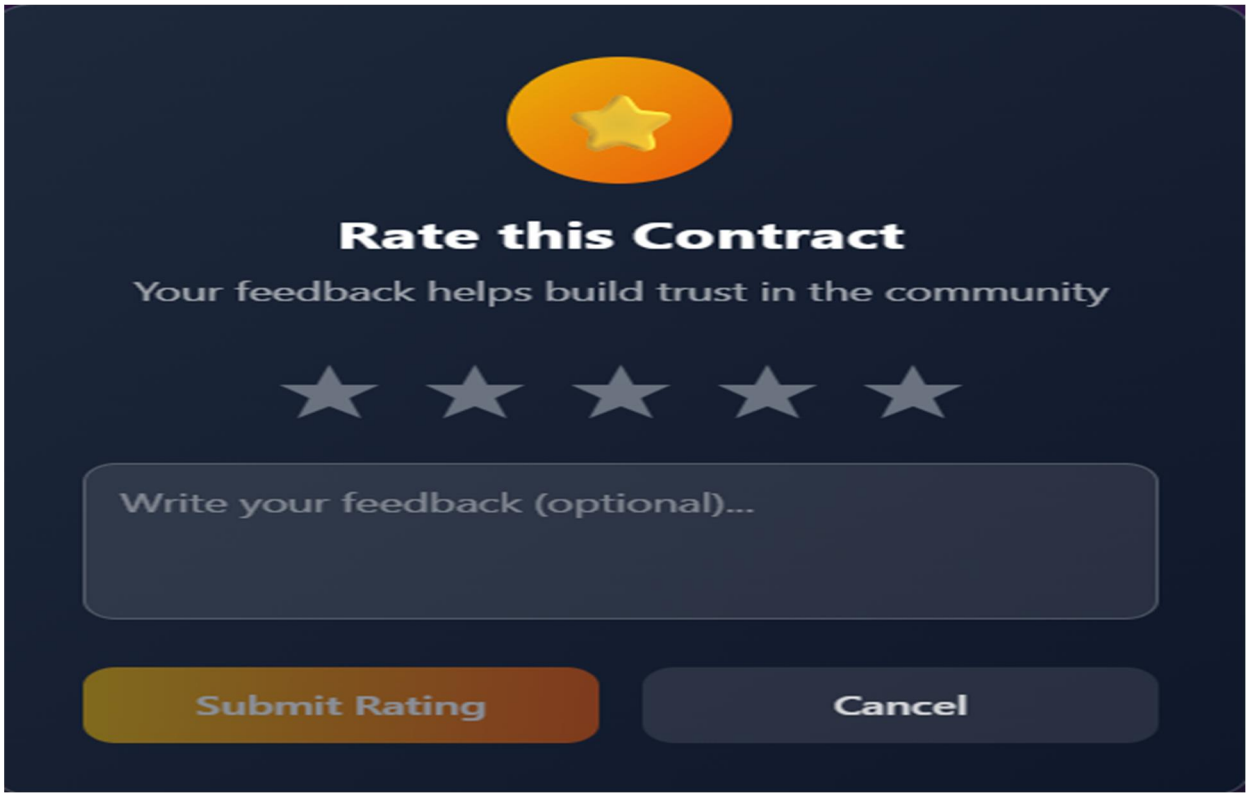
الشكل (18-5) واجهة عقود المستخدم

الوجهة الناتجة عن زر ViewDetails



الشكل (5-19) واجهة تفاصيل العقد

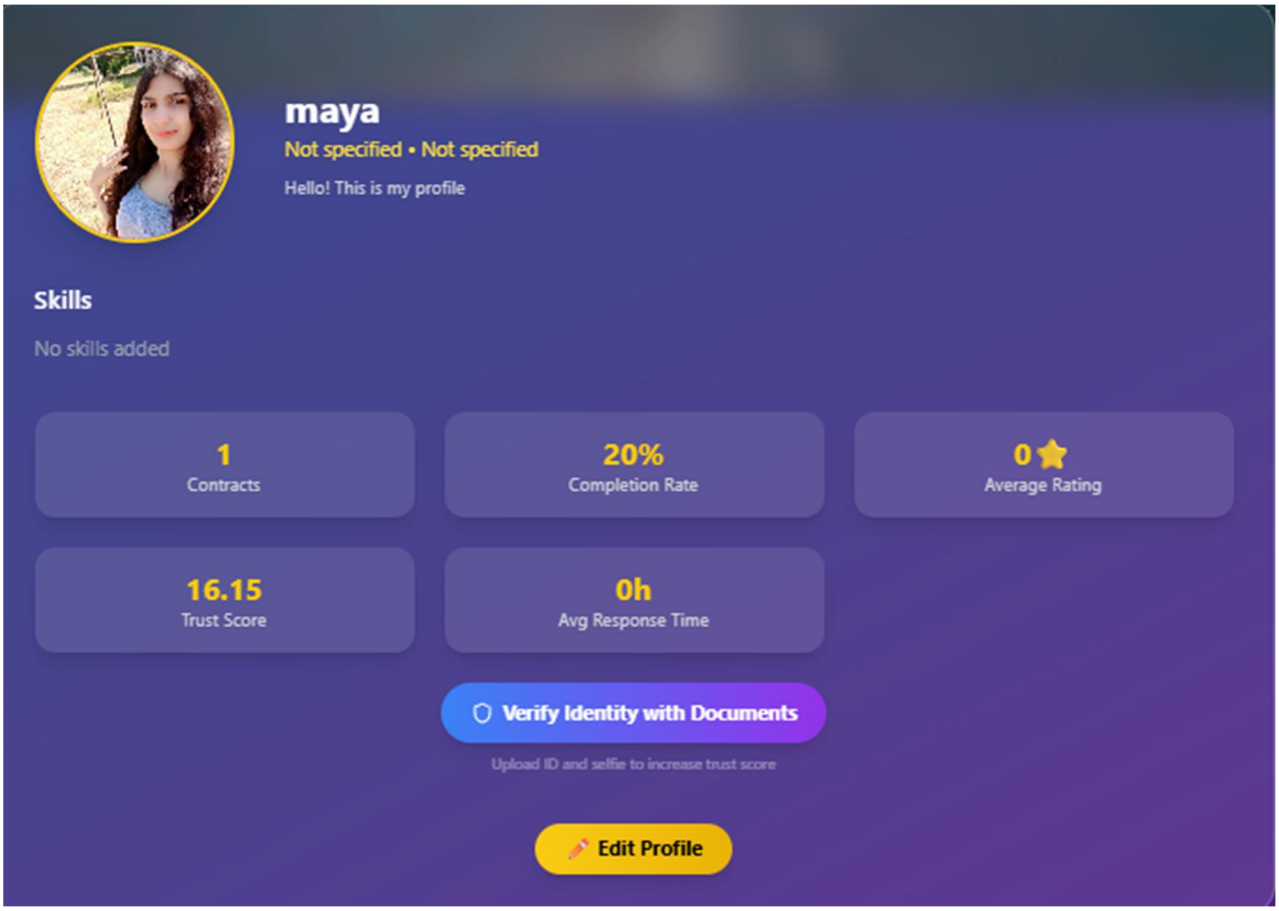
بعد اكمال العقد أي الضغط على زر completeيظهر زر تقييم العقد Rate



الشكل (5-20) واجهة تقييم العقد

الان سنعود لواجهة الdashbordلنرى الزر الثاني ألى أين يأخذنا الزر الثاني هو profileهذا الزر ياخذنا إلى الملف الشخصي

واجهة الملف الشخصي(Profile) :



الشكل (21-5) واجهة الملف الشخصي

الخاصية	الوصف
الغرض	إدارة الملف الشخصي وتوثيق الهوية
المستخدمون	جميع المستخدمين المسجلين
المبادئ المحققة	رفع ملفات، تعديل معلومات، توثيق الهوية

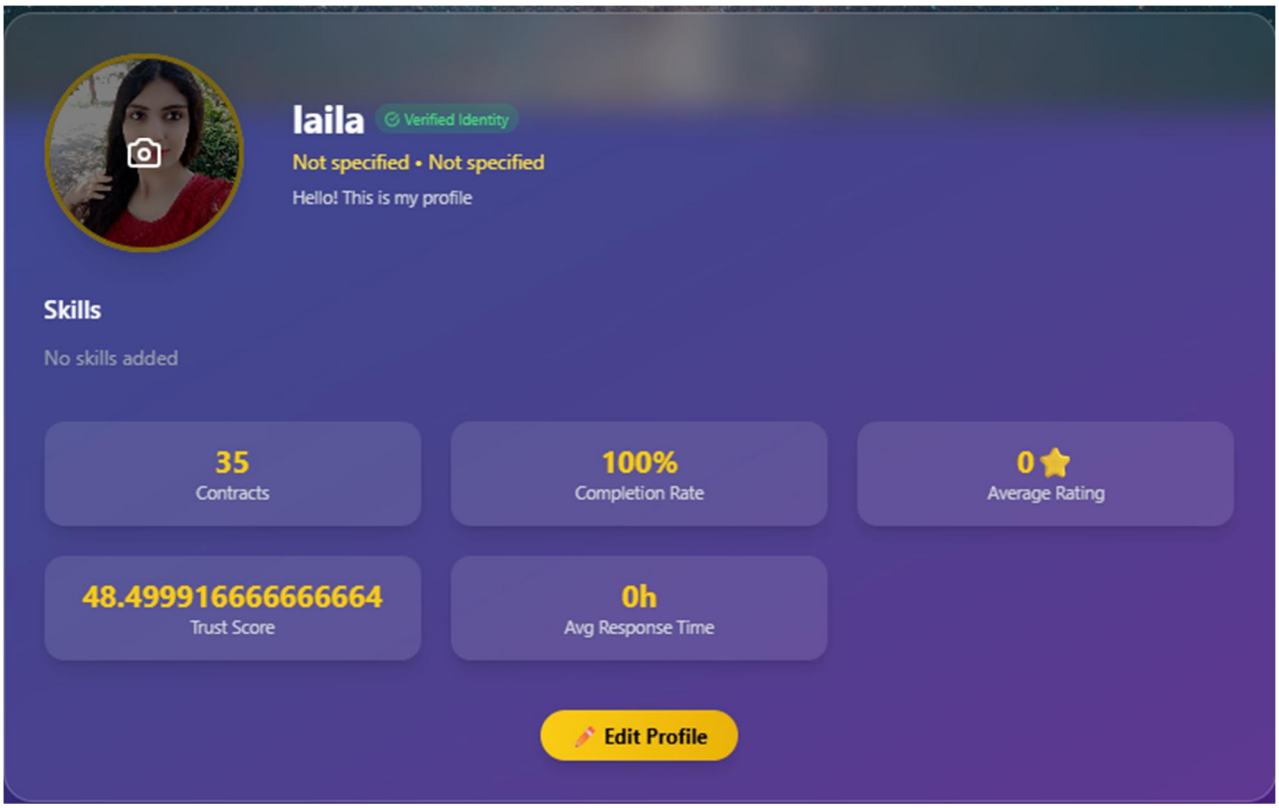
محتويات الواجهة:

- صورة الغلاف (Cover Image) قابلة للتعديل
- الصورة الشخصية (Profile Image) قابلة للتعديل
- معلومات المستخدم:
  - اسم المستخدم

- المدينة والبلد
- السيرة الذاتية(Bio)
- المهارات

- إحصائيات المستخدم:
  - عدد العقود
  - نسبة الإنجاز
  - متوسط التقييم
  - درجة الثقة
- زر "تعديل الملف الشخصي" (يفتح نافذة تعديل)
- زر "توثيق الهوية" (لرفع الوثائق الرسمية)
- زر ال verify identity with doucuments يظهر عندما يكون المستخدم لم يقوم بتوثيق حسابه بعد

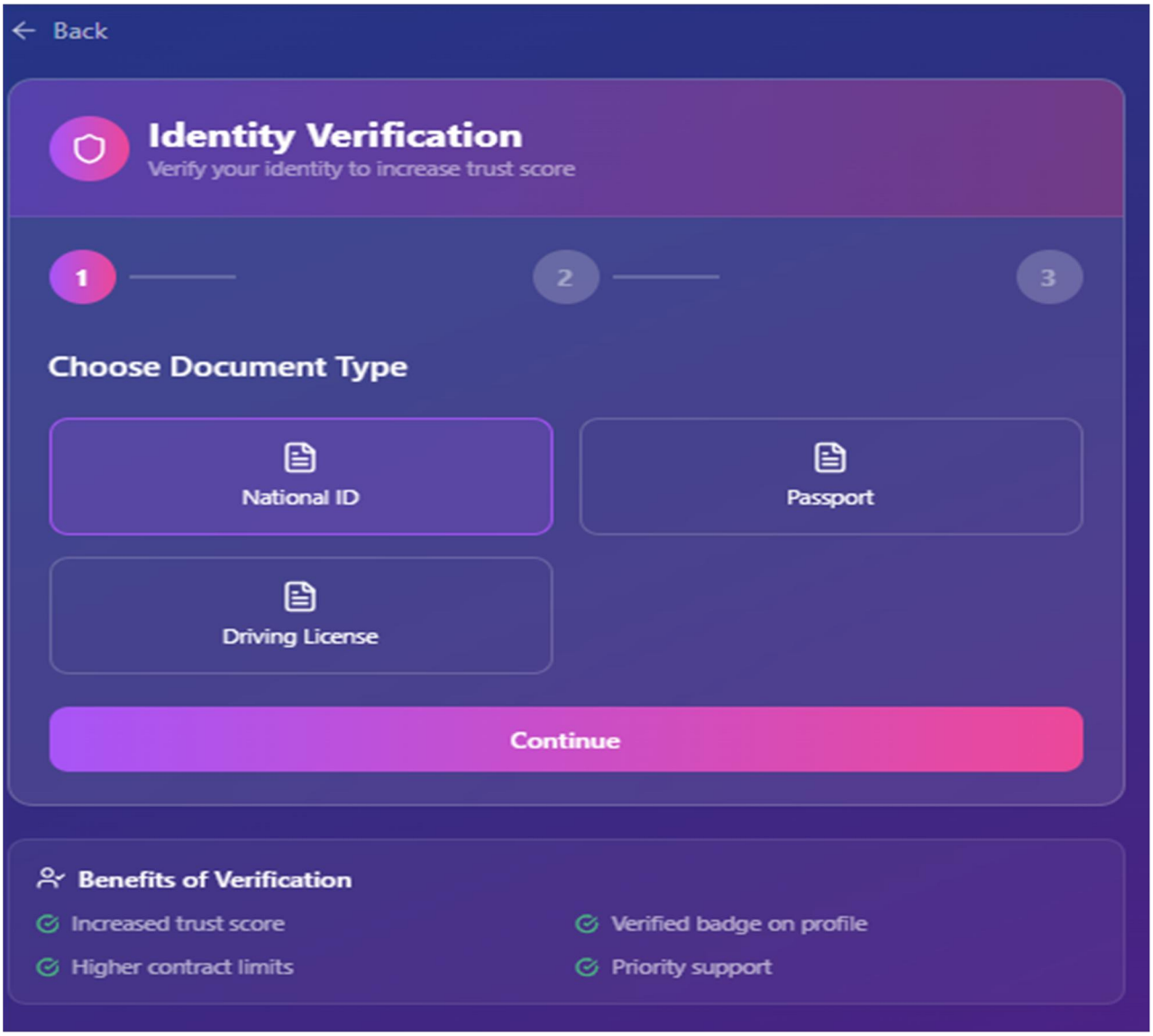
وحالما قام المستخدم بتوثيق هويتك سواء بصورة الهوية او جواز السفر أو شهادة السواقة فسوف يختفي هذا الزر عند توثيق الحساب وسنلاحظ ظهور رسالة خضراء جانب صورة الملف الشخصي كتأكيد على أنه تم توثيق الحساب بنجاح سنستعرض ذلك في الصورة التالية



الشكل (22-5) واجهة توضيحية لنجاح توثيق الحساب

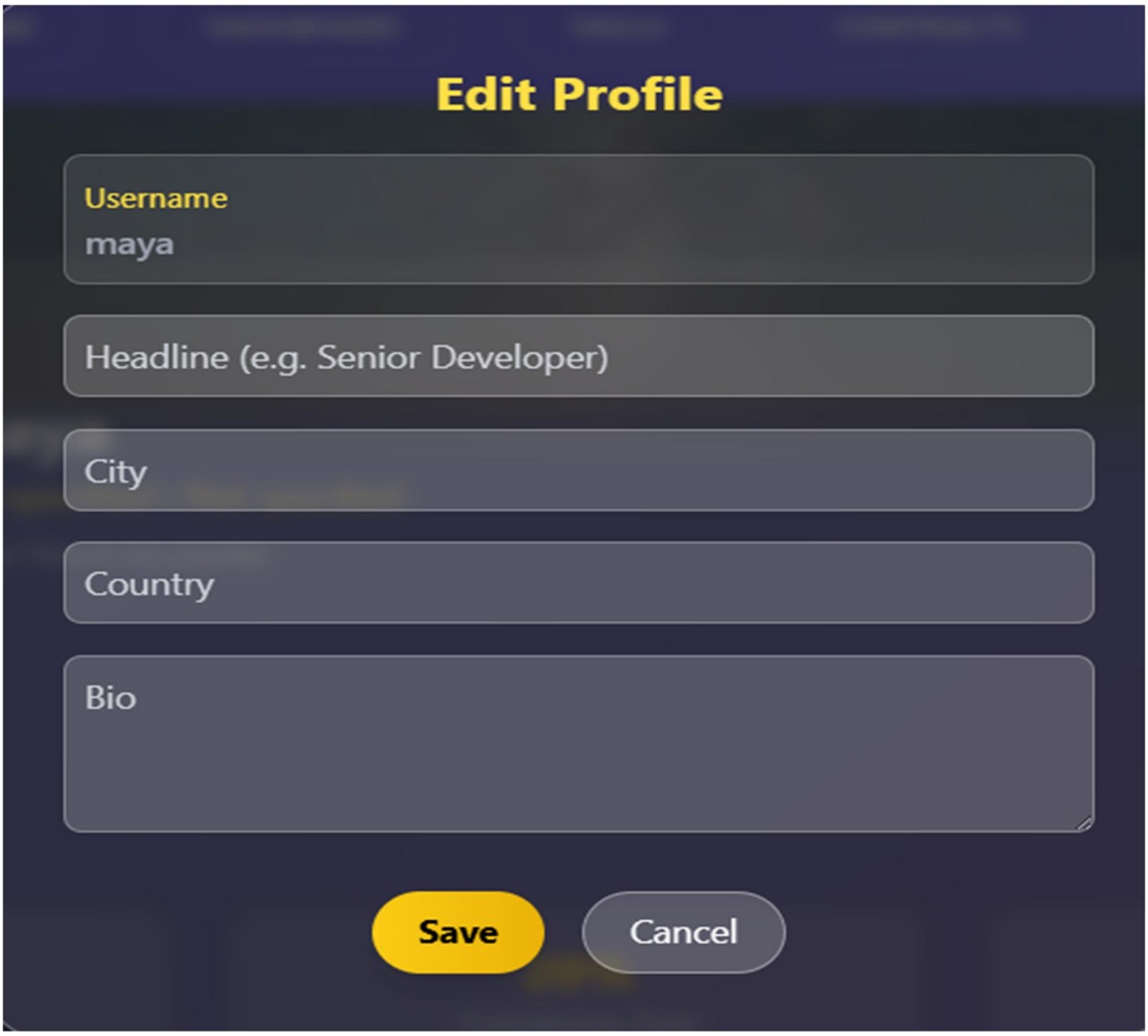
نلاحظ ان الزر المسؤول عن توثيق الحساب اختفى وظهرت رسالة تأكيد نجاح بالقرب من صورة الملف

الان سنستعرض الواجهة المسؤولة عن توثيق الهوية



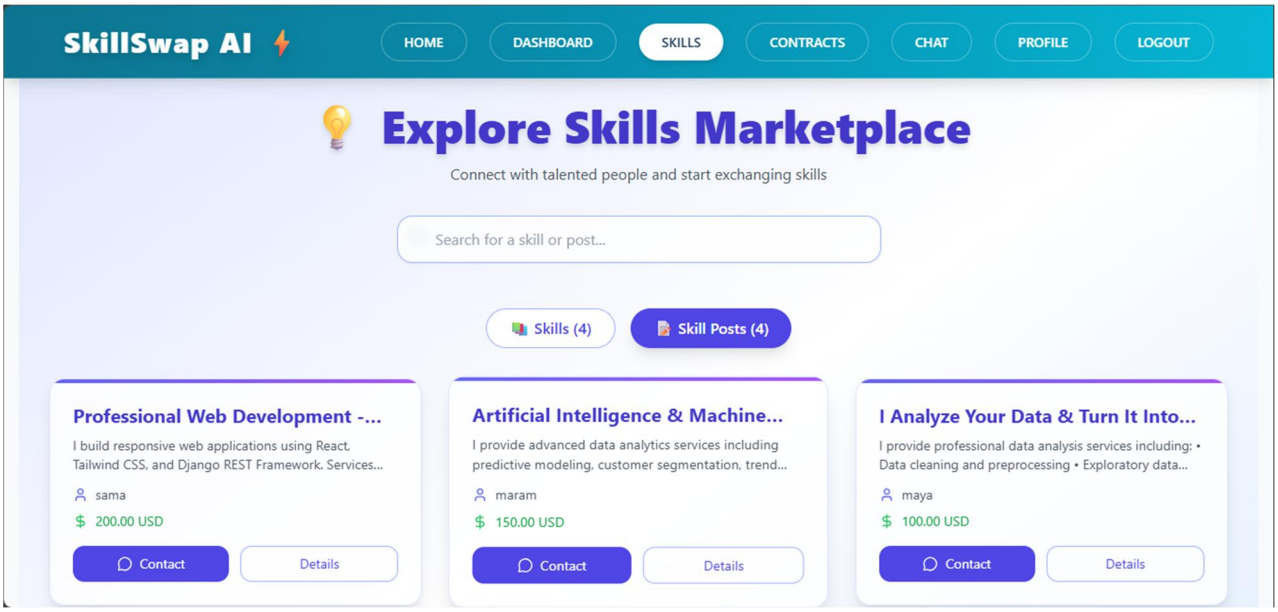
الشكل (23-5) واجهة توثيق الهوية

واجهة تعديل الملف الشخصي edit :



الشكل (24-5) واجهة تعديل الملف الشخصي

واجهة المنشورات والمهارات :



الشكل (25-5) واجهة المهارات

الخاصية	الوصف
الغرض	عرض والبحث عن المهارات والمنشورات المتاحة للتبادل
المستخدمون	المستخدمون المسجلون
المبادئ المحققة	بحث سريع، تصفية نتائج، عرض شبكي منظم

محتويات الواجهة:

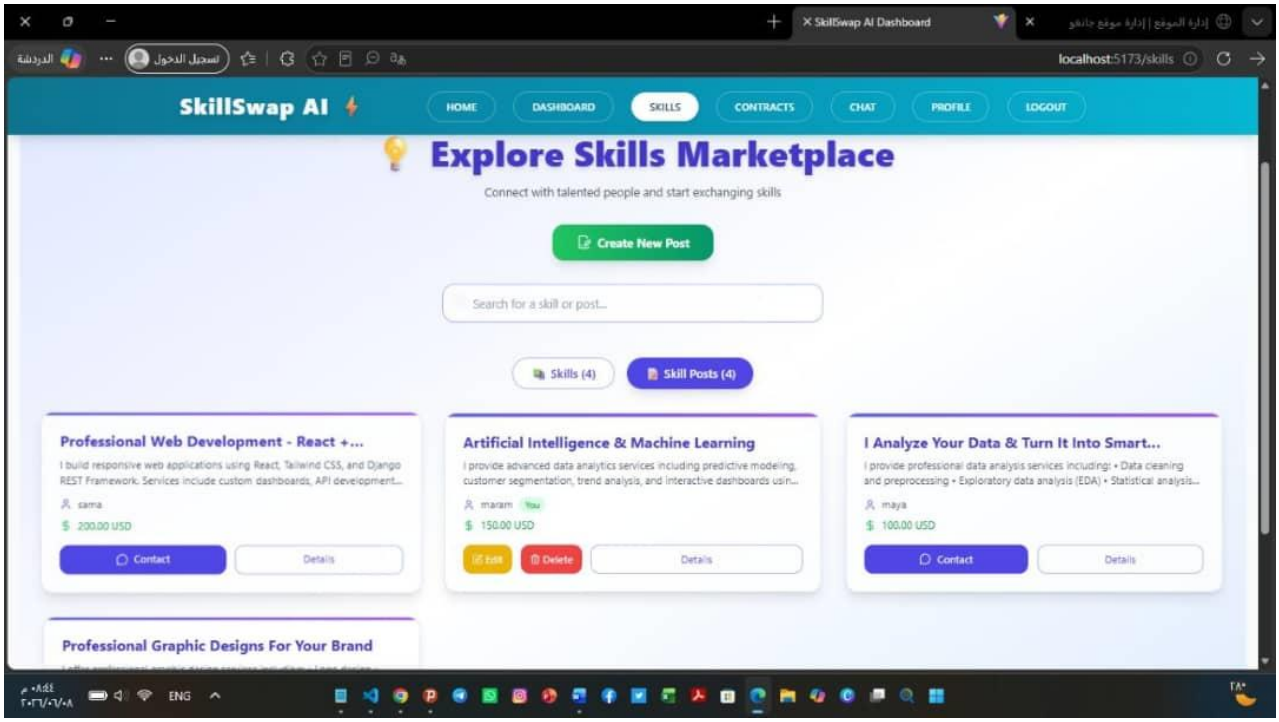
- شريط بحث للبحث عن مهارات أو منشورات
- أزرار تصفية(Skills / Posts)
- شبكة من البطاقات تعرض:
  - عنوان المنشور
  - وصف مختصر
  - صاحب المنشور
  - السعر
  - الموقع (إن وجد)

- زر "تواصل" لكل منشور

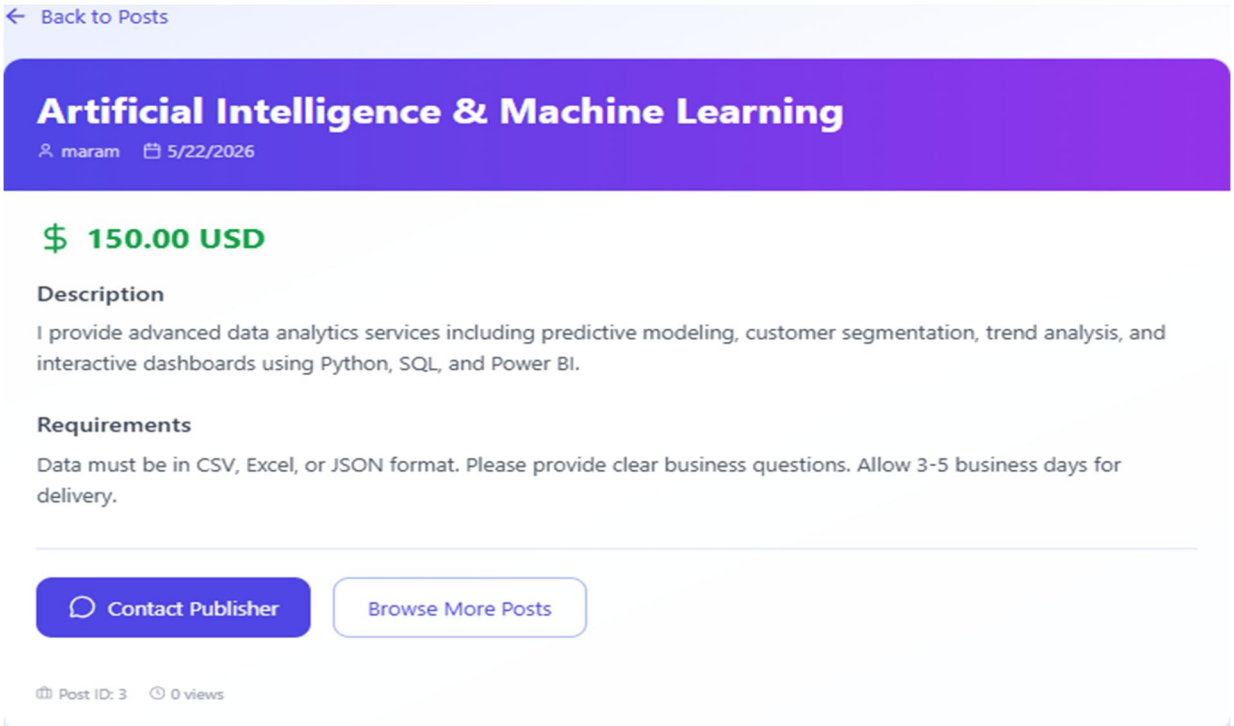
زر "تفاصيل" لكل منشور

في واجهة المنشورات نلاحظ كل منشور له زرين زر التواصل contact وزر تفاصيل لمنشور details

زر التواصل يؤدي إلى أنشاء غرفة دردشة بين المستخدم وناشر المنشور

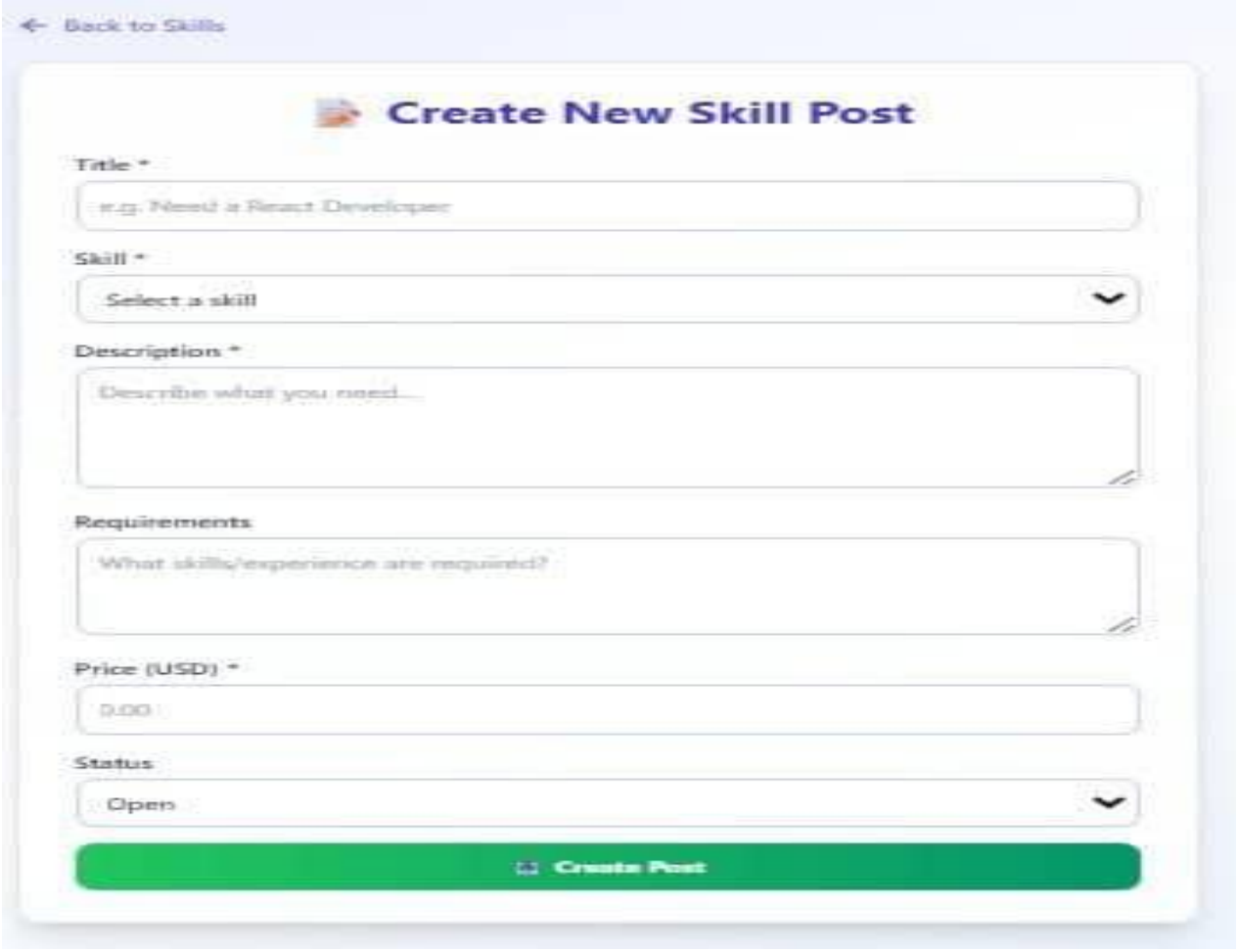


الشكل (26-5) واجهة المنشورات



الشكل (27- 5) واجهة تفاصيل المنشور

نلاحظ أن المستخدم المسجل دخول يمكنه تعديل منشوراته فقط فمثلا هنا تم تسجيل دخول بحساب maram فنلاحظ ظهور زرين حذف وتعديل بدل من تواصل لأنه لا يمكن إنشاء غرفة دردشة مع الشخص نفسه ولأن هذا المنشور خاص بالمستخدم المسجل



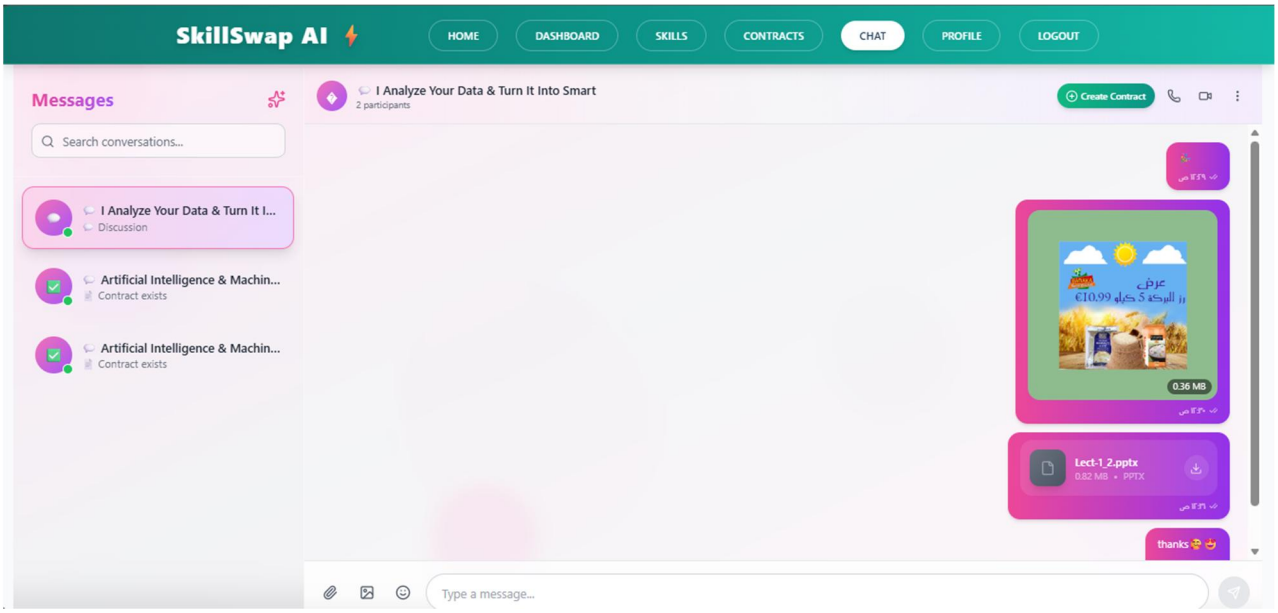
الشكل (28-5) واجهة إنشاء منشور

ننتقل إلى هذه الواجهة creat عند الضغط على زر



الشكل (5-29) واجهة تعديل المنشور

واجهة غرفة الدردشة:



الشكل (5-30) واجهة غرفة الدردشة

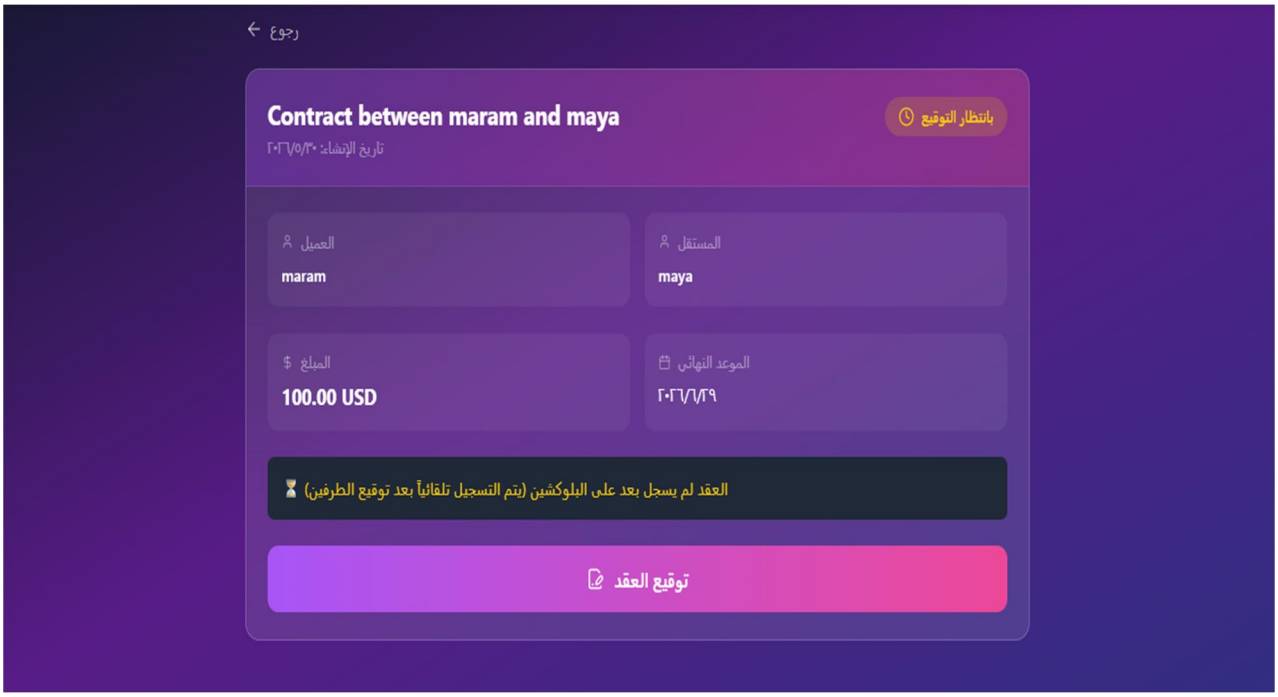
الخاصية	الوصف
الغرض	تواصل فوري بين المستخدمين للتفاوض على العقود
المستخدمون	المستخدمون الذين تواصلوا مع بعضهم البعض
المبادئ المحققة	اتصال فوري (WebSocket)، مؤشر كتابة، إشعارات القراءة

محتويات الواجهة

- قائمة المحادثات (الجهة اليسرى)
- منطقة عرض الرسائل (الوسط)
- حقل إدخال النص
- أزرار لإرسال:
  - رسالة نصية
  - صور
  - ملفات
  - إيموجي
- مؤشر "يكتب(Typing Indicator) "...
- إشعارات "مقروءة(Read Receipts) "
- زر "إنشاء عقد" (لإنشاء عقد من داخل المحادثة)

سنستعرض في الصورة التالية المحادثة بين طرفين حيث بالإمكان رفع صور وملفات وارسال الرسائل وكذلك الايموجي ونلاحظ وجود زر إنشاء العقد creat contract بعد ان يتم التفاوض بين الطرفين يتم إنشاء العقد ويتطلب ادخال المبلغ والمدة المقررة وبعدها يتحول الزر ألى تفاصيل العقد وتظهر رسالة ضمن المحادثة تؤكد انه تم انشاء العقد بنجاح

واجهة تفاصيل العقد :



الشكل (31-5) واجهة تفاصيل العقد

الخاصية	الوصف
الغرض	عرض تفاصيل العقد وحالة التوثيق على البلوكشين
المستخدمون	العميل والمستقل المرتبطان بالعقد
المبادئ المحققة	عرض معلومات واضحة، ربط مع أنظمة خارجية (Etherscan)

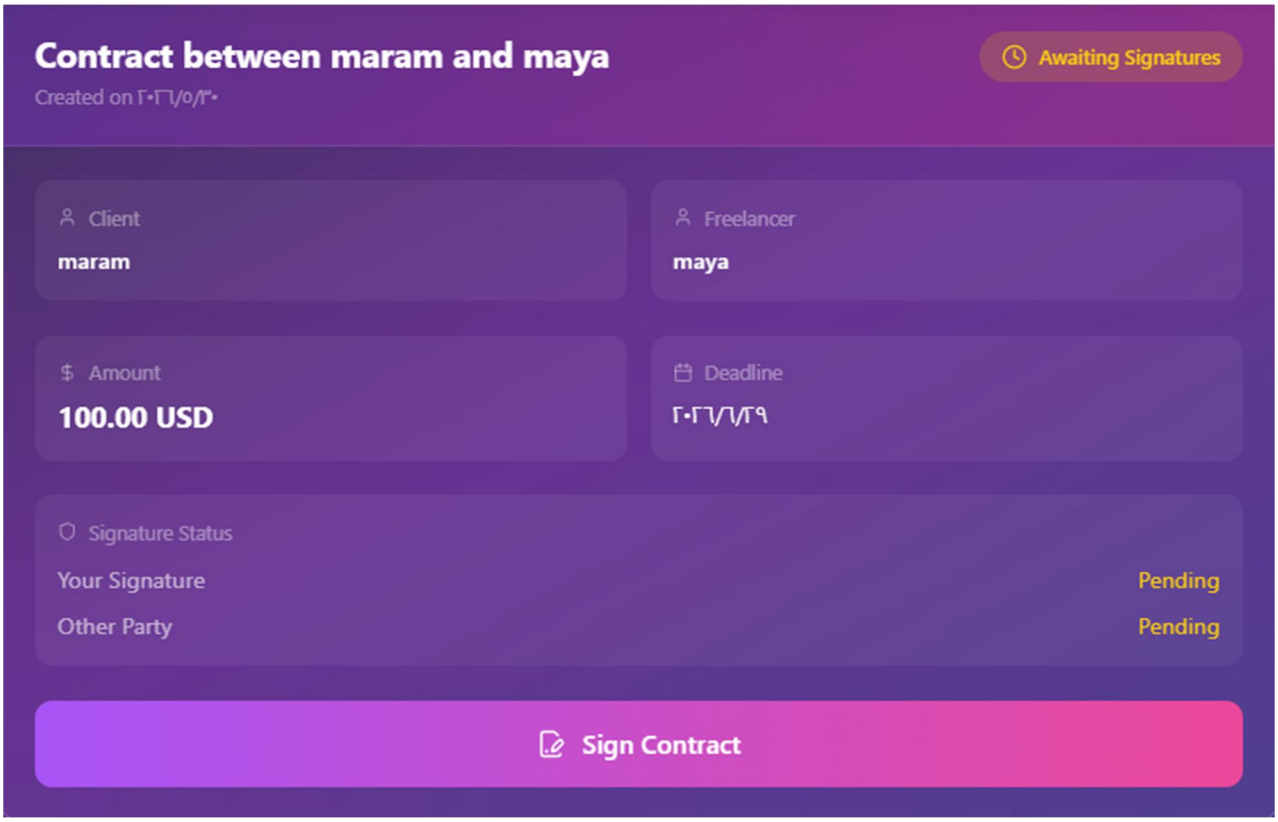
محتويات الواجهة:

- معلومات العقد:
  - رقم العقد
  - الأطراف (عميل / مستقل)
  - المبلغ
  - المدة
  - الحالة(pending / active / completed)
- حالة التوقيعات (مُوقّع / لم يُوقّع)

- حالة البلوكشين (مُسجّل / لم يُسجّل)
- رابط إلى Etherscan إن وجد
- زر "تقييم العقد" (بعد الإكمال)

عند الضغط على زر توقيع العقد ياخذنا إلى واجهة التحقق من هوية المستخدم اما برمز الPINأو بالبصمة

واجهة إنشاء وتوقيع العقد :



الشكل (32-5) واجهة إنشاء وتوقيع العقد

الخاصية	الوصف
الغرض	إنشاء عقد جديد أو توقيع عقد قائم
المستخدمون	العميل (Client) والمستقل(Freelancer)
المبادئ المحققة	التحقق من الهوية، رسائل فورية، إمكانية تصحيح الخطأ

محتويات الواجهة:

- تفاصيل العقد (المبلغ، المدة، الشروط)
- حالة التوقيع (تم توقيع الطرف الأول / الثاني)
- زر "توقيع العقد"
- عند الضغط على زر التوقيع:
  - تظهر نافذة منبثقة للتحقق PIN) أو بصمة(
  - بعد التحقق، يتم توقيع العقد
  - تظهر رسالة تأكيد

عندما اضغط على هذا الزر سيفتح واجهة التحقق من هوية المستخدم أما برمز الPINأو يالبصمة



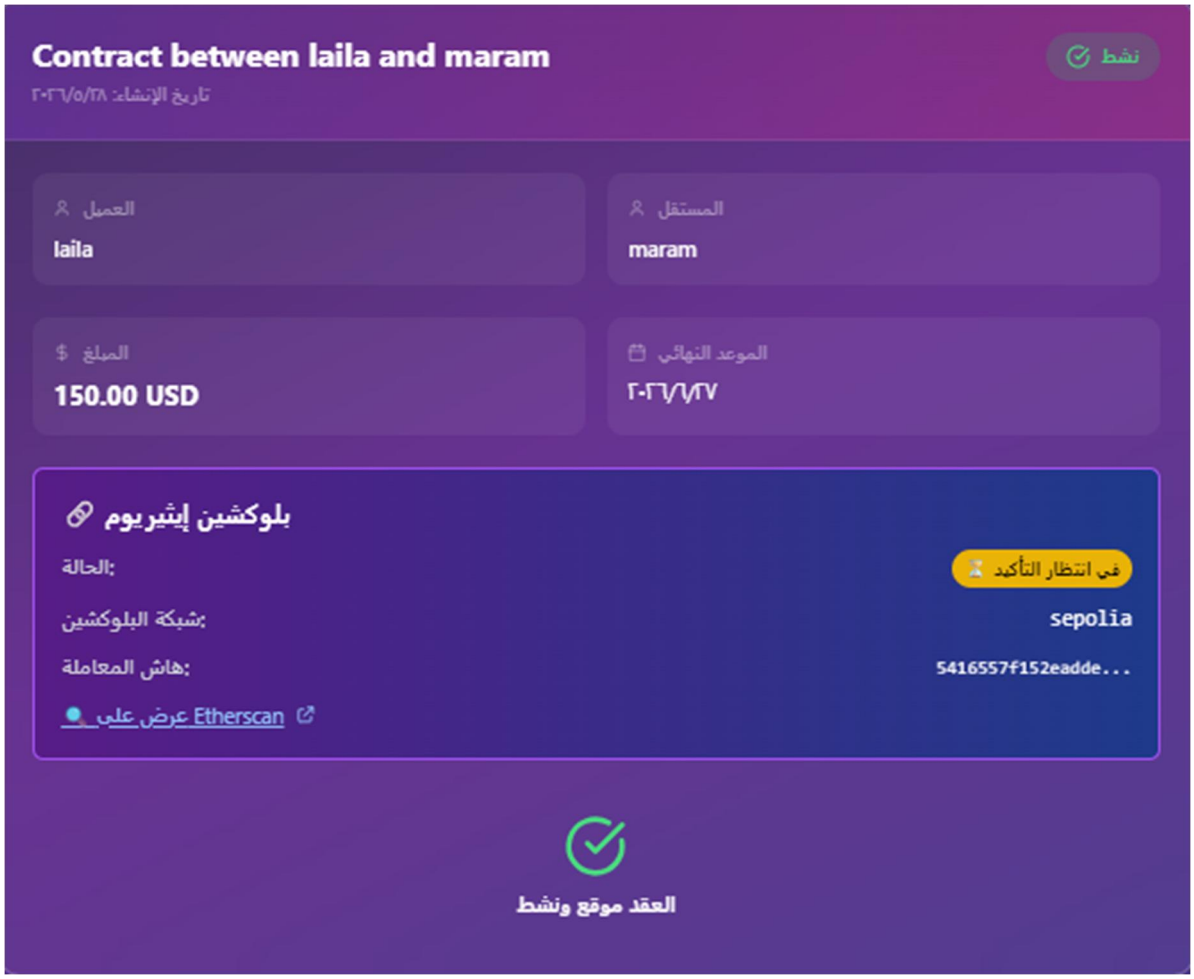
الشكل (5-33) واجهة التحقق من الهوية بالبصمة البيومترية

وهذه الواجهة التالية هي واجهة متقدمة تظهر فقط الوسيلة البيومترية التي سجلها المستخدم عندما قام بإنشاء حساب



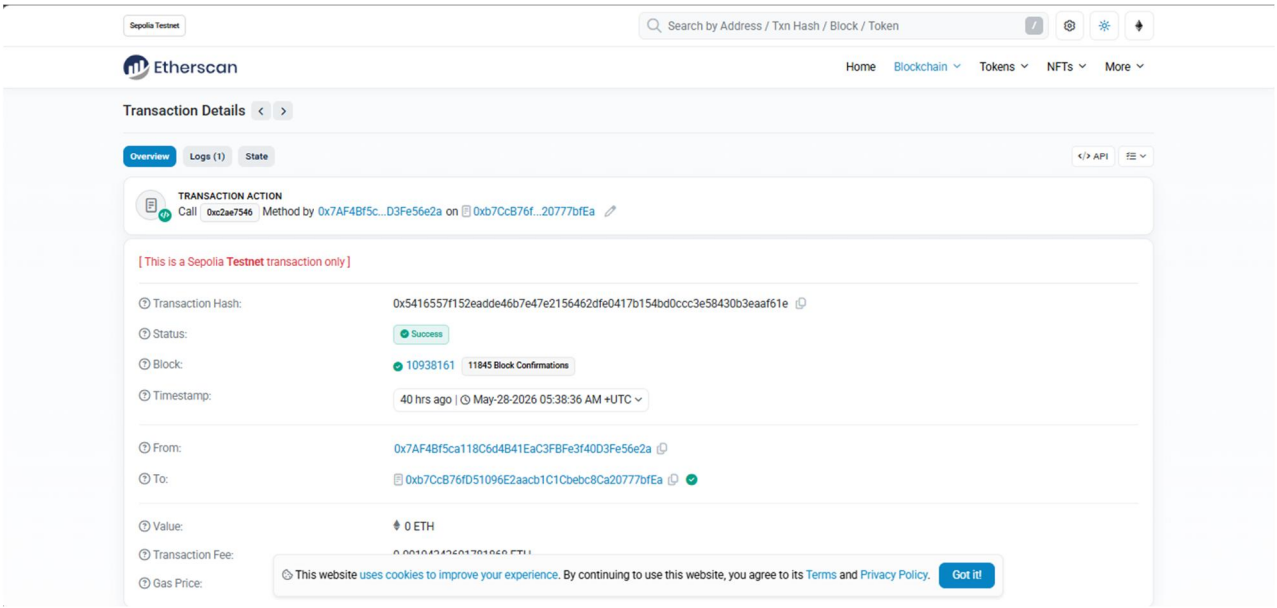
الشكل (5-34) واجهة تحقق تكيفية

عندما يقوم الطرفين بتوقيع العقد سيتم تخزينه بشكل تلقائي على البلوكتشين سنوضح ذلك في الصورة سنرى كيف عندما قام الطرفين بالتوقيع كيف تخزن العقد على البلوكتشين



الشكل (35-5) واجهة اكتمال العقد

عندما نضغط على الرابط سنرى كيف تم حفظ العقد على البلوكتشين



الشكل (36-5) واجهة التسجيل على البلوكشين

الفصل السادس :
التحديات والاقتراحات المستقبلية

Chapter Six :

Challenges And Futuer Proposals

التحديات التي واجهتنا أثناء تنفيذ مشروعنا :

اعتمادية بيئة التشغيل والأجهزة (Hardware & Environment Dependency) :

تتطلب المصادقة البيومترية وجود كاميرات وميكروفونات ذات جودة مناسبة chez المستخدمين.

تأثر دقة التمييز البيومتري بالعوامل الخارجية (مثل ضعف الإضاءة عند التقاط الوجه، أو الضوضاء المحيطة أثناء تسجيل الصوت)

غياب التطبيق المخصص للهواتف :

تقتصر المنصة حالياً على واجهات الويب، مما يحد من سهولة الاستخدام السريعة عبر الهواتف الذكية

الاقتراحات المستقبلية لتطوير مشروعنا :

تطوير تطبيق للهواتف الذكية

للاستفادة المباشرة من المستشعرات المدمجة (Flutter أو React Native باستخدام) بناء تطبيق موبايل في الهواتف (مثل حساس البصمة وكاميرا الهاتف) لتحسين تجربة المستخدم

تعزيز آليات كشف التزييف الحيوي

وحركات الوجه (Eye Blinking) دمج نماذج ذكاء اصطناعي أكثر تقدماً للكشف عن الطرف العيني

.العشوائية وتغيرات النبرة الصوتية لضمان وجود الشخص الحقيقي أثناء الجلسة

المراجع :

1. SpeechBrain: A General-Purpose Speech Toolkit (2021). https://speechbrain.github.io/
2. Buterin, V. (2014). Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.
3. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. (FAISS paper)
4. Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems.
5. Django Software Foundation. Django Documentation. https://docs.djangoproject.com/
6. Meta Platforms. React Documentation. https://react.dev/
7. Web3.py Documentation. https://web3py.readthedocs.io/